FUNDAÇÃO EDSON QUEIROZ
UNIVERSIDADE DE FORTALEZA — UNIFOR
CENTRO DE CIÊNCIAS TECNOLÓGICAS
CURSO DE CIÊNCIA DA COMPUTAÇÃO

FLEET MANAGER: SISTEMA WEB DE GESTÃO DE FROTAS

André Luiz Cavalcante da Silva (2310287)
Luiz Eduardo Pacheco (2310314)
Gregory Figueiredo de M. P. Jereissati (2320425)

Fortaleza
2026

André Luiz Cavalcante da Silva
Luiz Eduardo Pacheco
Gregory Figueiredo de M. P. Jereissati

André Luiz Cavalcante da Silva

Luiz Eduardo Pacheco

Gregory Figueiredo de M. P. Jereissati

FLEET MANAGER: SISTEMA WEB DE GESTÃO DE FROTAS

Trabalho apresentado ao Curso de Ciência da Computação do Centro de Ciências Tecnológicas da Universidade de Fortaleza, como requisito do Projeto Final Integrador I.

Aprovado em: ____/____/_____.

FLEET MANAGER: SISTEMA WEB DE GESTÃO DE FROTAS

Trabalho apresentado ao Curso de Ciência da Computação do Centro de Ciências Tecnológicas da Universidade de Fortaleza, como requisito do Projeto Final Integrador I.

Orientador: Prof. Me. Ronaldo Gonçalves Junior.

BANCA EXAMINADORA

_____

Prof. Me. Ronaldo Gonçalves Junior (Orientador)
Universidade de Fortaleza

_____

Avaliador(a)

Fortaleza

2026

Avaliador(a)

RESUMO

A gestão de frotas em organizações de pequeno e médio porte costuma ser conduzida de forma descentralizada, distribuída entre planilhas isoladas e controles manuais. A informação existe, porém fragmentada, e o acompanhamento de prazos é passivo: documentos vencem sem que ninguém seja avisado e o custo de cada veículo não é apurado. Este trabalho apresenta o Fleet Manager, aplicação web que centraliza o controle operacional, financeiro e documental de frotas em uma base de dados única, organizada em torno do veículo. O sistema foi construído em TypeScript no servidor e na interface, em um *monorepo* gerenciado por npm workspaces: a interface é uma *Single Page Application* em React com Vite, e a interface de programação de aplicações REST, em Node.js com Express, adota arquitetura em camadas com dependência unidirecional entre rotas, controladores, serviços e repositórios. A persistência ocorre em PostgreSQL gerenciado pelo Supabase, acessado por SQL parametrizado, sem mapeador objeto-relacional. A autenticação é delegada ao Supabase Auth, que emite *tokens* assinados com chave assimétrica pelo algoritmo ES256; a aplicação os verifica com a chave pública publicada pelo próprio serviço, sem armazenar segredo algum. O controle de acesso baseado em papéis permanece sob responsabilidade da aplicação, com papel e situação de aprovação persistidos fora do *token*, o que confere efeito imediato ao bloqueio de contas. Os dados são segregados por empresa: o identificador da organização participa da condição de toda consulta e deriva do perfil autenticado, nunca de dado enviado pelo cliente. Situações de prazo — documento vencido, manutenção atrasada, habilitação vencida — são calculadas a cada consulta e reunidas em uma central de alertas. A qualidade foi verificada por quatrocentos e dois testes automatizados em três níveis, unitário, integração com o banco e ponta a ponta, executados em integração contínua a cada mudança e cobrindo 88,4% das linhas do servidor, e por testes de carga, volume e resiliência: em um cenário misto com proporção realista de leituras e escritas, a interface de programação atendeu de 790 a 1.452 requisições por segundo, sem erro, com até cem conexões simultâneas. O sistema encontra-se publicado em ambiente de produção, com interface e servidor atendidos pelo mesmo domínio. Registra-se, como delimitação, que a validação empregou base de dados fictícia, não tendo o sistema sido aplicado em uma organização real.

Palavras-chave: gestão de frotas; arquitetura de software; aplicações web; controle de acesso baseado em papéis; computação em nuvem.

ABSTRACT

Fleet management in small and medium-sized organizations is often carried out in a decentralized way, spread across isolated spreadsheets and manual controls. The information exists, but it is fragmented, and deadline tracking is passive: documents expire without anyone being warned, and the cost of each vehicle is not calculated. This work presents Fleet Manager, a web application that centralizes the operational, financial and documentary control of fleets in a single database organized around the vehicle. The system was built in TypeScript on both the server and the user interface, in a monorepo managed with npm workspaces: the interface is a Single Page Application in React with Vite, and the REST application programming interface, in Node.js with Express, adopts a layered architecture with one-way dependencies among routes, controllers, services and repositories. Data is persisted in PostgreSQL managed by Supabase and accessed through parameterized SQL, without an object-relational mapper. Authentication is delegated to Supabase Auth, which issues tokens signed with an asymmetric key using the ES256 algorithm; the application verifies them with the public key published by the service itself, without storing any secret. Role-based access control remains the responsibility of the application, with role and approval status stored outside the token, which makes account blocking take effect immediately. Data is segregated by company: the organization identifier is part of the condition of every query and is derived from the authenticated profile, never from data sent by the client. Deadline situations — expired document, overdue maintenance, expired driver license — are computed on every query and gathered in an alert center. Quality was verified by four hundred and two automated tests at three levels, unit, database integration and end-to-end, run in continuous integration on every change and covering 88.4% of the server lines, and by load, volume and resilience tests: in a mixed scenario with a realistic proportion of reads and writes, the application programming interface served from 790 to 1,452 requests per second, without errors, with up to one hundred concurrent connections. The system is deployed in a production environment, with the interface and the server served from the same domain. As a limitation, validation used a fictitious database, and the system has not been applied in a real organization.

Keywords: fleet management; software architecture; web applications; role-based access control; cloud computing.

LISTA DE FIGURAS

Figura 1 — Visão geral da solução..... 22
Figura 2 — Casos de uso significativos para a arquitetura..... 23
Figura 3 — Camadas da aplicação e sentido das dependências..... 24
Figura 4 — Organização em pacotes do *monorepo*..... 25
Figura 5 — Modelo entidade-relacionamento..... 26
Figura 6 — Autenticação e acesso a recurso protegido..... 28
Figura 7 — Ciclo de vida da conta de usuário..... 29
Figura 8 — Painel de indicadores com um ano de operação fictícia..... 32
Figura 9 — Central de alertas, com o painel de notificações do cabeçalho aberto..... 33
Figura 10 — Anexo de arquivo ao documento..... 34
Figura 11 — Diagrama de implantação..... 36
Figura 12 — Integração e entrega contínuas..... 37

LISTA DE QUADROS

Quadro 1 — Tecnologias adotadas por camada.....20
Quadro 2 — Camadas de verificação do sistema.....21
Quadro 3 — Entidades do modelo de dados.....26
Quadro 4 — Matriz de controle de acesso por papel.....30
Quadro 5 — Requisitos funcionais e situação de atendimento.....39
Quadro 6 — Requisitos não funcionais e situação de atendimento.....40
Quadro 7 — Regras de negócio implementadas e motivo de cada uma.....40
Quadro 8 — Verificações executadas sobre o código.....42
Quadro 9 — Testes de ponta a ponta por área.....43
Quadro 10 — Objetivos específicos, entregas e evidências.....49
Quadro 11 — Cronograma resumido.....56
Quadro 12 — Partes interessadas do projeto.....57
Quadro 13 — Requisitos e restrições arquiteturais.....59

LISTA DE TABELAS

Tabela 1 — Cobertura de código do servidor.....44
Tabela 2 — Workload misto por nível de concorrência.....45
Tabela 3 — Workload misto por rota.....46
Tabela 4 — Carga por rota isolada.....46
Tabela 5 — Estresse do painel de indicadores.....47
Tabela 6 — Tempos com 50 mil despesas em uma empresa.....47
Tabela 7 — Carga em produção.....48
Tabela 8 — Comportamento com o banco fora do ar e na volta.....48

LISTA DE ABREVIATURAS E SIGLAS

API	*Application Programming Interface* (interface de programação de aplicações)
CD	*Continuous Delivery* (entrega contínua)
CI	*Continuous Integration* (integração contínua)
CNH	Carteira Nacional de Habilitação
CPF	Cadastro de Pessoas Físicas
CRLV	Certificado de Registro e Licenciamento de Veículo
E2E	*End-to-End* (de ponta a ponta)
ES256	ECDSA sobre a curva P-256 com SHA-256
HTTP	*HyperText Transfer Protocol*
IPVA	Imposto sobre a Propriedade de Veículos Automotores
JSON	*JavaScript Object Notation*
JWKS	*JSON Web Key Set*
JWT	*JSON Web Token*
ORM	*Object-Relational Mapper* (mapeador objeto-relacional)
RBAC	*Role-Based Access Control*
REST	*Representational State Transfer*
RF	Requisito Funcional
RLS	*Row Level Security* (segurança em nível de linha)
RN	Regra de Negócio
RNF	Requisito Não Funcional
SPA	*Single Page Application*
SQL	*Structured Query Language*
URL	*Uniform Resource Locator*

SUMÁRIO

1 INTRODUÇÃO..... 12
1.1 Justificativa..... 12
1.2 Objetivo geral..... 13
1.3 Objetivos específicos..... 13
1.4 Estrutura do trabalho..... 14
2 FUNDAMENTAÇÃO TEÓRICA..... 15
2.1 Gestão de frotas..... 15
2.2 Aplicações web e arquitetura cliente-servidor..... 15
2.3 Arquitetura REST e interface de programação..... 16
2.4 Banco de dados relacional e acesso a dados..... 16
2.5 Autenticação delegada e controle de acesso..... 17
2.6 Validação de dados de entrada..... 18
2.7 Teste de software, integração contínua e desempenho..... 18
3 METODOLOGIA..... 19
3.1 Ciclo de desenvolvimento..... 19
3.2 Versionamento e modelagem..... 20
3.3 Ferramentas e ambiente..... 20
3.4 Estratégia de verificação..... 21
4 DESENVOLVIMENTO..... 22
4.1 Casos de uso..... 22
4.2 Arquitetura em camadas..... 23
4.3 Organização do repositório..... 24
4.4 Modelo de dados..... 25
4.5 Autenticação e controle de acesso..... 28
4.6 Módulos funcionais e interface..... 30
4.7 Armazenamento de arquivos..... 33
4.8 Rotina automatizada de vencimentos..... 35
4.9 Proteção do banco de dados..... 35
4.10 Publicação em produção..... 36
4.11 Integração e entrega contínuas..... 37
4.12 Observabilidade e tratamento de falhas..... 38
4.13 Delimitação do escopo..... 38

5 RESULTADOS E DISCUSSÕES..... 39
5.1 Requisitos e regras de negócio..... 39
5.2 Verificação funcional..... 42
5.3 Segurança..... 44
5.4 Desempenho..... 45
5.5 Resiliência..... 48
5.6 Atendimento dos objetivos específicos..... 48
5.7 Limitações e discussão..... 49
6 CONCLUSÃO..... 51
REFERÊNCIAS..... 53
GLOSSÁRIO..... 55
APÊNDICE A — Termo de abertura do projeto..... 56
A.1 Identificação do projeto..... 56
A.2 Objetivos..... 56
A.3 Escopo..... 56
A.4 Cronograma resumido..... 56
A.5 Partes interessadas..... 57
A.6 Restrições..... 57
A.7 Riscos e mitigação..... 58
A.8 Critérios de sucesso..... 58
APÊNDICE B — Documento de arquitetura de software..... 59
B.1 Requisitos e restrições arquiteturais..... 59
B.2 Visão de casos de uso..... 59
B.3 Visão lógica..... 60
B.4 Visão de implementação..... 60
B.5 Visão de processo..... 60
B.6 Visão de implantação..... 60
B.7 Visão de dados..... 61
APÊNDICE C — Roteiro para reproduzir os testes..... 62

1 INTRODUÇÃO

A gestão de frotas é atividade estratégica para organizações que dependem do uso contínuo de veículos em suas operações. Ela envolve simultaneamente três dimensões distintas e interdependentes: a operacional, que trata de quais veículos existem, em que estado se encontram e quais manutenções foram realizadas ou estão previstas; a financeira, que apura quanto cada veículo consome em combustível, peças, serviços, multas, tributos e seguros; e a documental, que acompanha os documentos obrigatórios de veículos e condutores e suas respectivas datas de vencimento.

Essas três dimensões descrevem o mesmo ativo — o veículo — mas, na prática das organizações, costumam ser administradas por pessoas diferentes, em ferramentas diferentes e em momentos diferentes. O setor financeiro registra as despesas em uma planilha; o responsável pela operação anota as manutenções em outro lugar; a documentação permanece arquivada sem qualquer mecanismo de acompanhamento de prazos. O resultado é uma operação em que nenhum ponto isolado da organização enxerga o veículo por inteiro.

Convém observar que o problema não reside na ausência de informação: os dados normalmente existem. Ele decorre da fragmentação dessa informação e da ausência de um mecanismo ativo de acompanhamento de prazos. Uma planilha registra que um documento vence em determinada data, porém não avisa ninguém quando essa data se aproxima, e o vencimento só é percebido quando já produziu consequência.

Nesse contexto, o Fleet Manager é proposto como aplicação web voltada à centralização das informações operacionais, financeiras e documentais de veículos em uma base de dados relacional única. O presente documento descreve o sistema tal como foi efetivamente construído: cada afirmação técnica aqui registrada corresponde a código verificado no repositório do projeto.

1.1 Justificativa

Este trabalho justifica-se pela necessidade de reduzir os problemas decorrentes da gestão de frotas conduzida de maneira manual ou descentralizada. O uso de planilhas isoladas dificulta o acompanhamento preciso de despesas, manutenções e vencimentos, o que compromete a eficiência operacional e amplia riscos financeiros e legais.

A relevância do problema manifesta-se em consequências concretas: multas por documentação vencida, decorrentes do controle passivo de prazos; substituição da

manutenção preventiva pela corretiva, mais onerosa e associada à indisponibilidade do veículo; e custo operacional que permanece invisível, por não ser apurado por ativo. O projeto mostra-se pertinente do ponto de vista acadêmico, por exercitar arquitetura de software, modelagem de dados e segurança de aplicações, e do ponto de vista prático, por endereçar necessidade real de organizações de pequeno e médio porte.

1.2 Objetivo geral

Desenvolver um sistema web para gestão de frotas, com foco no controle operacional, financeiro e documental de veículos, promovendo transparência e rastreabilidade dos custos por ativo, classificação automática de vencimentos e redução de riscos legais.

Delimita-se o alcance: o sistema consolida e classifica dados registrados, mas não realiza projeção financeira nem recomendação automática. Os indicadores apresentados são apurações do histórico lançado, e a classificação de vencimentos deriva de regra declarada sobre a data de validade — não de inferência.

1.3 Objetivos específicos

- Desenvolver funcionalidades de cadastro, edição e gerenciamento de veículos, motoristas e usuários do sistema, incluindo o vínculo entre motoristas e veículos.
- Permitir o registro de despesas obrigatoriamente vinculadas a um veículo, contemplando as categorias combustível, manutenção, multa, IPVA, seguro e outras.
- Implementar o controle de manutenções preventivas e corretivas, com acompanhamento dos estados programada, realizada, atrasada e cancelada, sendo o atraso apurado automaticamente a partir da data prevista.
- Registrar documentos obrigatórios de veículos e motoristas, com data de vencimento, classificação automática da situação e anexo do arquivo digital correspondente, e reunir em uma central de alertas os prazos que exigem ação: documentos, manutenções e habilitações.
- Disponibilizar painel de indicadores que apresente custo total, custo médio, custo por veículo, evolução mensal das despesas e distribuição por categoria.
- Implementar autenticação de usuários e controle de acesso baseado em papéis, assegurando segregação de responsabilidades no uso do sistema.

- Verificar a qualidade do software por meio de testes automatizados em três níveis — unitário, integração com o banco e ponta a ponta —, executados em integração contínua, e avaliar a segurança, o desempenho e a resiliência do sistema.

1.4 Estrutura do trabalho

O capítulo 2 apresenta a fundamentação teórica que embasa as decisões adotadas. O capítulo 3 descreve a metodologia de desenvolvimento. O capítulo 4 detalha a construção do sistema, incluindo arquitetura, modelo de dados, segurança e publicação. O capítulo 5 expõe os resultados obtidos e discute as limitações identificadas. O capítulo 6 conclui o trabalho. O glossário define os termos técnicos de uso restrito. Os apêndices reúnem o termo de abertura do projeto, o documento de arquitetura de software e o roteiro para reproduzir os testes.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo apresenta os fundamentos que embasam o desenvolvimento do Fleet Manager, abordando os conceitos de gestão de frotas, as tecnologias adotadas e os padrões de software empregados na construção do sistema.

2.1 Gestão de frotas

A gestão de frotas compreende o conjunto de atividades voltadas ao planejamento, à organização, ao controle e ao monitoramento dos veículos utilizados por uma organização. O processo envolve o acompanhamento de custos operacionais, a manutenção preventiva e corretiva, o gerenciamento da documentação obrigatória e o monitoramento do desempenho dos condutores. Quando realizada de forma manual ou por meio de planilhas isoladas, essa gestão está sujeita a erros humanos, inconsistências e dificuldades no acompanhamento histórico (Fleury; Wanke; Figueiredo, 2000).

Sistemas informatizados superam tais limitações ao centralizar as informações e automatizar processos críticos, como a classificação de vencimentos, o cálculo de indicadores financeiros e o controle do estado das manutenções. A informatização reduz o risco de sanções por documentação irregular, melhora a previsibilidade orçamentária e apoia a decisão estratégica.

2.2 Aplicações web e arquitetura cliente-servidor

Sistemas de informação baseados na web são acessados por navegadores e fundamentam-se na arquitetura cliente-servidor, na qual o cliente responde pela interface com o usuário e o servidor pelo processamento das requisições e pela gestão dos dados. Segundo Sommerville (2011), esses sistemas permitem acesso ubíquo e facilitam a manutenção centralizada do software, o que os torna escolha natural para aplicações corporativas.

Categoria relevante de aplicação web é a *Single Page Application*, que carrega uma única página e atualiza o conteúdo dinamicamente, sem recarregamento integral. A interface do Fleet Manager adota a biblioteca React em conjunto com a linguagem TypeScript, cuja tipagem estática antecipa para o tempo de compilação erros que, de outro modo, só se manifestariam em execução. A construção emprega o Vite e a estilização utiliza o *framework*

utilitário Tailwind CSS. O sistema oferece internacionalização em português brasileiro e inglês por meio da biblioteca i18next.

2.3 Arquitetura REST e interface de programação

REST é um estilo arquitetural para sistemas distribuídos proposto por Fielding (2000), que emprega o protocolo HTTP e seus métodos para operar sobre recursos identificados por localizadores uniformes. Uma interface RESTful promove a separação de responsabilidades entre a camada de apresentação e a de serviço, viabilizando o desenvolvimento independente de cada uma.

No Fleet Manager, a interface de programação é construída em Node.js com o *framework* Express e organizada em camadas — rotas, *middlewares*, controladores, serviços e repositórios — com dependência unidirecional. Essa separação não é ornamental: por não conhecerem o protocolo HTTP, os serviços podem ser exercitados por testes automatizados sem que seja necessário iniciar servidor ou banco de dados, o que viabiliza a verificação descrita na seção 5.2.

2.4 Banco de dados relacional e acesso a dados

O PostgreSQL é sistema gerenciador de banco de dados relacional de código aberto, reconhecido pela robustez, pela conformidade com o padrão SQL e pelo suporte a transações com garantias de atomicidade, consistência, isolamento e durabilidade. Tais características asseguram a integridade referencial em cenários com múltiplas entidades relacionadas, como veículos, motoristas, despesas e documentos.

O acesso aos dados ocorre por SQL parametrizado, sem mapeador objeto-relacional. A consulta é escrita na linguagem do próprio banco, e os valores trafegam como parâmetros da instrução, nunca concatenados ao texto dela — condição que impede a injeção de SQL por construção, e não por disciplina de quem escreve.

A escolha tem contrapartida. Um mapeador gera cliente tipado a partir de esquema declarativo e dispensa a escrita manual de consultas rotineiras; abrir mão dele custa verbosidade. Em troca, o comportamento no banco fica explícito no código: não há tradução intermediária entre o que se lê e o que é executado, e recursos do PostgreSQL permanecem disponíveis sem depender do que a camada de abstração expõe. Considerando que o

isolamento entre organizações depende de o identificador da empresa participar da condição de toda consulta, a legibilidade direta dessa condição pesou na decisão.

O banco é hospedado no Supabase, plataforma que provê PostgreSQL gerenciado. O acesso a partir da aplicação é restrito à camada de repositórios.

A estrutura do banco evolui por *migrations*: arquivos SQL numerados, cada um com uma alteração do esquema, aplicados em ordem por um executor que registra quais já rodaram. O mesmo conjunto de arquivos reconstrói o banco do zero em qualquer ambiente — o de desenvolvimento, o de testes e o de produção —, o que torna o esquema parte do código versionado, e não um estado mantido à mão no servidor.

2.5 Autenticação delegada e controle de acesso

A autenticação verifica a identidade de um usuário, ao passo que a autorização determina quais recursos esse usuário pode acessar. São responsabilidades distintas, e o Fleet Manager as atribui a agentes distintos.

A autenticação é delegada ao Supabase Auth. As credenciais deixam de ser responsabilidade da aplicação, que não armazena senha alguma. O serviço emite *tokens* no padrão JSON Web Token (Jones; Bradley; Sakimura, 2015), assinados com chave assimétrica pelo algoritmo ES256. A aplicação verifica cada *token* utilizando a chave pública publicada pelo próprio serviço em *endpoint* JWKS, conferindo assinatura, emissor e público. Decorre desse desenho uma propriedade relevante: a aplicação não guarda nenhum segredo do serviço de identidade, de modo que não há chave compartilhada a proteger ou rotacionar.

A autorização, por sua vez, permanece sob controle da aplicação e segue o modelo de controle de acesso baseado em papéis. Sandhu *et al.* (1996) definem o RBAC como mecanismo eficaz para a administração de permissões em sistemas corporativos, por associar permissões a papéis e usuários a papéis, reduzindo a complexidade da administração de segurança. O sistema adota três papéis: administrador, com acesso irrestrito; gestor, responsável pela administração da frota; e operador, com permissões restritas ao registro da rotina operacional.

Cabe destacar uma decisão de projeto: o papel de acesso e a situação de aprovação do usuário são persistidos na base da aplicação, e não transportados no *token*. O *token* não constitui, portanto, fonte de autoridade sobre permissões. A consequência prática é que o bloqueio de uma conta produz efeito imediato, sem depender da expiração da credencial em posse do usuário.

2.6 Validação de dados de entrada

A validação dos dados de entrada é prática essencial à integridade e à segurança das informações. O Zod é biblioteca de validação e análise de esquemas para TypeScript que permite declarar, de forma explícita, a estrutura esperada dos dados recebidos. Sua integração ao encadeamento de *middlewares* do Express possibilita rejeitar requisições malformadas antes que alcancem a camada de negócio.

2.7 Teste de software, integração contínua e desempenho

Sommerville (2011) distingue o teste de desenvolvimento em níveis: o de unidade, que exercita uma parte isolada; o de componente, que verifica a integração entre partes; e o de sistema, que verifica o conjunto montado. Cada nível encontra uma classe de defeito que os outros não alcançam: o teste de unidade aponta a regra que mudou, mas não vê uma consulta errada no banco; o de sistema vê o comportamento final, mas não diz onde está a falha. No Fleet Manager, os três níveis correspondem aos testes unitários dos serviços, aos testes de integração contra o PostgreSQL e aos testes de ponta a ponta, que enviam requisições HTTP reais à interface de programação. Todos são escritos com o *framework* Vitest; os de ponta a ponta usam também a biblioteca Supertest.

A integração contínua consiste em executar automaticamente a compilação e os testes a cada alteração enviada ao repositório, de modo que um defeito seja detectado no momento em que é introduzido; a entrega contínua estende a prática para que toda versão aprovada esteja pronta para publicação (Humble; Farley, 2010). O sistema adota as duas, encadeadas: a produção só recebe a versão que passou em todos os testes, conforme a seção 4.11.

O desempenho é caracterizado por duas grandezas: a vazão, quantidade de requisições atendidas por segundo, e o tempo de resposta, expresso por percentis. A mediana (p50) é o tempo dentro do qual metade das respostas chegou; o p99, aquele dentro do qual chegaram 99% delas, e revela a regularidade do serviço. Como critério de adequação, adota-se o limite de um segundo apontado por Nielsen (1993) como o tempo máximo de resposta que preserva o fluxo de raciocínio do usuário. A norma ISO/IEC 25010 (International Organization for Standardization, 2011) classifica esses aspectos sob as características de eficiência de desempenho e de confiabilidade, esta última incluindo a capacidade de recuperação após falha.

3 METODOLOGIA

O desenvolvimento do Fleet Manager baseou-se em abordagem ágil e incremental, com entregas organizadas em ciclos e validação contínua do escopo.

3.1 Ciclo de desenvolvimento

O trabalho foi dividido em etapas incrementais, cada uma dedicada a um conjunto específico de funcionalidades. A etapa inicial tratou da configuração da arquitetura, da padronização de código e da modelagem do banco de dados. As etapas seguintes contemplaram, sucessivamente, autenticação e controle de acesso; cadastro de veículos e motoristas; registro de despesas e manutenções; gestão de documentos com anexo de arquivos; painel de indicadores e administração de usuários; publicação em ambiente de produção; segregação de dados por organização; e, por fim, uma etapa dedicada à verificação — testes de ponta a ponta, integração contínua, observabilidade e testes de carga — seguida de revisão das regras de negócio à luz do que os testes mostraram.

Ao longo do processo, decisões arquiteturais foram revistas com base em evidência prática, e o registro dessas revisões integra o resultado do trabalho. A autenticação, inicialmente delegada a um provedor de identidade externo e em seguida implementada pela própria aplicação, foi por fim atribuída ao Supabase Auth, consolidando autenticação, persistência e armazenamento em um único serviço e eliminando a responsabilidade de guardar senhas. A plataforma de publicação do servidor também foi revista, conforme detalhado na seção 4.10, e o acesso aos arquivos anexados passou do navegador para a aplicação, conforme a seção 4.7.

A revisão final das regras de negócio comparou, item a item, o que este documento afirmava e o que o código fazia, e adotou em cada caso a versão mais completa. Três promessas do documento passaram a existir no sistema: a manutenção atrasada, antes apenas um valor possível, passou a ser apurada pela data prevista; a central de alertas, antes restrita a documentos, passou a reunir manutenções e habilitações; e a rotina diária passou a registrar quando sinalizou cada documento. Na mesma revisão, cada regra foi avaliada quanto ao benefício que traz a quem usa o sistema, o que resultou em três regras novas, descritas no Quadro 7.

3.2 Versionamento e modelagem

O controle de versão foi realizado com Git, e o código hospedado no GitHub. A modelagem precedeu a implementação e incluiu a definição dos casos de uso, do modelo entidade-relacionamento e dos fluxos de autenticação e autorização. O esquema do banco é versionado por sete *migrations* incrementais, de modo que a estrutura possa ser reconstruída a partir do repositório em um banco vazio. As *migrations* são somente de adição: o executor guarda uma soma de verificação de cada arquivo aplicado e recusa continuar se um deles for alterado depois, pois o banco e o histórico teriam divergido. Toda mudança de esquema entra, portanto, como arquivo novo.

3.3 Ferramentas e ambiente

O projeto adota organização em *monorepo* gerenciado por npm workspaces, reunindo as duas aplicações e um pacote de tipos compartilhados. Optou-se por npm workspaces, e não por ferramentas de orquestração de *monorepo* mais elaboradas, por adequação ao porte do projeto: o recurso já integra o gerenciador de pacotes e dispensa camada adicional de construção. A padronização de código emprega ESLint e Prettier. O Quadro 1 sintetiza as tecnologias adotadas em cada camada.

Quadro 1 — Tecnologias adotadas por camada

Camada	Tecnologia
Linguagem	TypeScript, no servidor e na interface
Interface	React 18, Vite, React Router 7, Tailwind CSS 4, Recharts, i18next
Servidor	Node.js com Express 4, em arquitetura de camadas
Acesso a dados	postgres.js — SQL parametrizado, sem ORM
Banco de dados	PostgreSQL gerenciado pelo Supabase
Autenticação	Supabase Auth, com verificação de *token* por JWKS
Armazenamento de arquivos	Supabase Storage
Validação	Zod 4
Agendamento	node-cron em ambiente local; agendador da plataforma em produção
Testes	Vitest e Supertest; gerador de carga próprio, versionado no repositório
Integração e entrega contínuas	GitHub Actions, com publicação na Vercel pela interface de linha de comando
Organização do repositório	*Monorepo* com npm workspaces
Publicação	Vercel, em projeto único

Fonte: Elaborado pelos autores (2026).

3.4 Estratégia de verificação

A verificação foi organizada em camadas, da mais barata e rápida à mais próxima do uso real. Cada camada responde a uma pergunta diferente sobre o sistema, conforme o Quadro 2; os resultados de cada uma estão no capítulo 5.

Quadro 2 — Camadas de verificação do sistema

Camada	Pergunta que responde	Como
Análise estática	O código está correto antes de rodar?	Compilação TypeScript, ESLint, construção e auditoria de dependências
Unitária	Cada regra de negócio faz o que deveria?	Vitest, com os repositórios substituídos por dublês
Integração	As consultas fazem no banco o que se espera?	Vitest contra o PostgreSQL, em transações revertidas
Ponta a ponta	As peças funcionam montadas, pelo protocolo HTTP?	Vitest e Supertest, com *tokens* reais do Supabase Auth
Integração contínua	Tudo isso roda sozinho a cada mudança?	GitHub Actions, com banco criado do zero
Carga e volume	O sistema aguenta muitos acessos e muitos dados?	Gerador de carga, com até 200 conexões e 50 mil despesas
Resiliência	Como o sistema reage se o banco cair?	Banco pausado com a interface de programação no ar
Produção	O sistema publicado responde como o testado?	Rotas, permissões, telas e carga moderada na Vercel

Fonte: Elaborado pelos autores (2026).

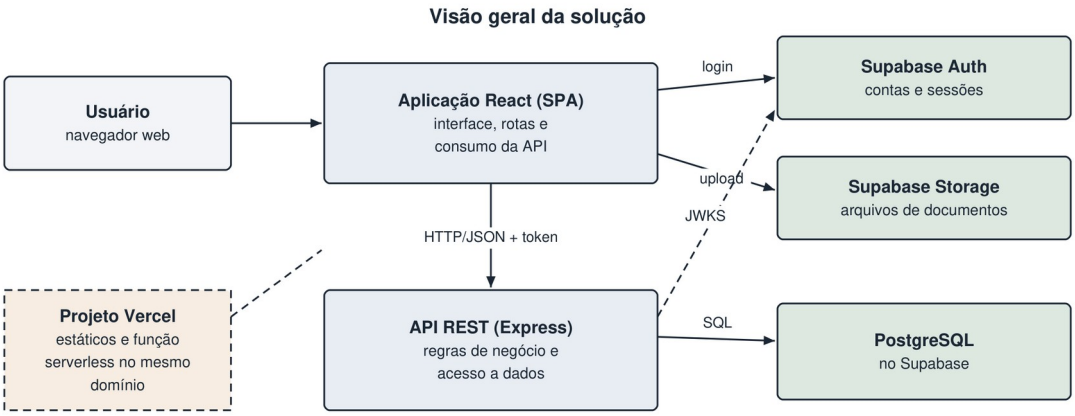
Para a demonstração e os testes de interface, um *script* acrescenta às duas empresas fictícias um ano de operação com datas relativas ao dia da execução: em cada uma, dezoito veículos, catorze motoristas, solicitações de acesso pendentes e cerca de 1.250 despesas, cem manutenções e sessenta documentos, com uma parte vencida ou atrasada. O *script* usa semente fixa, de modo que repetir a execução gera os mesmos dados; não altera o que já existe e pode ser desfeito por completo.

Os testes que gravam dados — os de ponta a ponta e os de carga — só executam contra o ambiente local, um Supabase completo em contêineres, isolado da produção; a suíte recusa iniciar se o endereço configurado não for local. Na produção, a verificação limitou-se a leituras e a uma carga moderada, para não alterar dados nem consumir a cota do plano gratuito.

4 DESENVOLVIMENTO

Este capítulo descreve a construção do sistema. A Figura 1 apresenta a visão geral da solução e situa os componentes descritos nas seções seguintes.

Figura 1 — Visão geral da solução



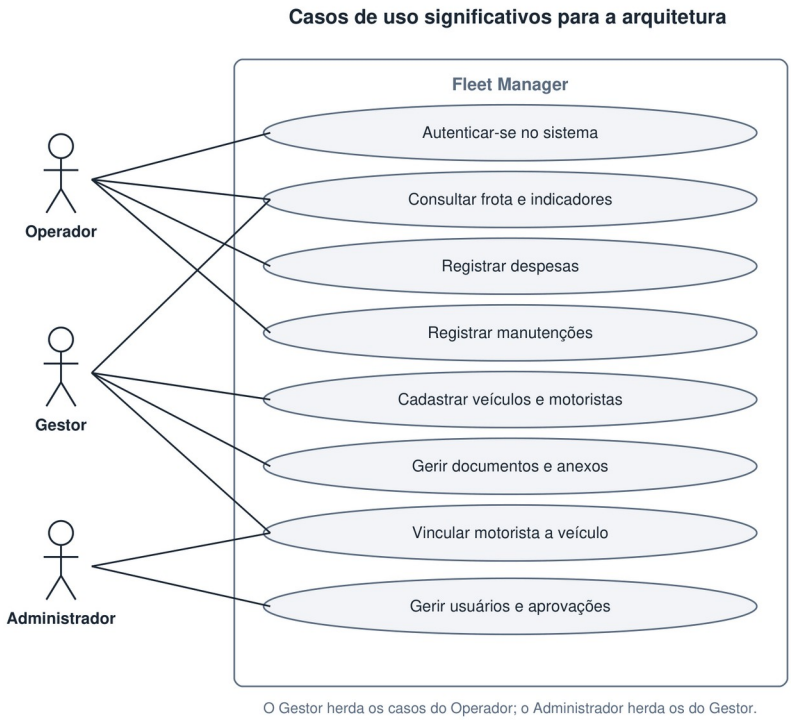
Fonte: Elaborado pelos autores (2026).

O Supabase cumpre três papéis independentes: autentica os usuários, hospeda o banco relacional e armazena os arquivos anexados aos documentos. Nem a autenticação nem o envio de arquivos transitam pela camada de serviço — implicações discutidas nas seções 4.5 e 4.7.

4.1 Casos de uso

A Figura 2 apresenta os casos de uso significativos para a arquitetura, organizados por papel. Os papéis são cumulativos: o gestor exerce os casos do operador, e o administrador exerce os do gestor, acrescidos da administração de usuários.

Figura 2 — Casos de uso significativos para a arquitetura

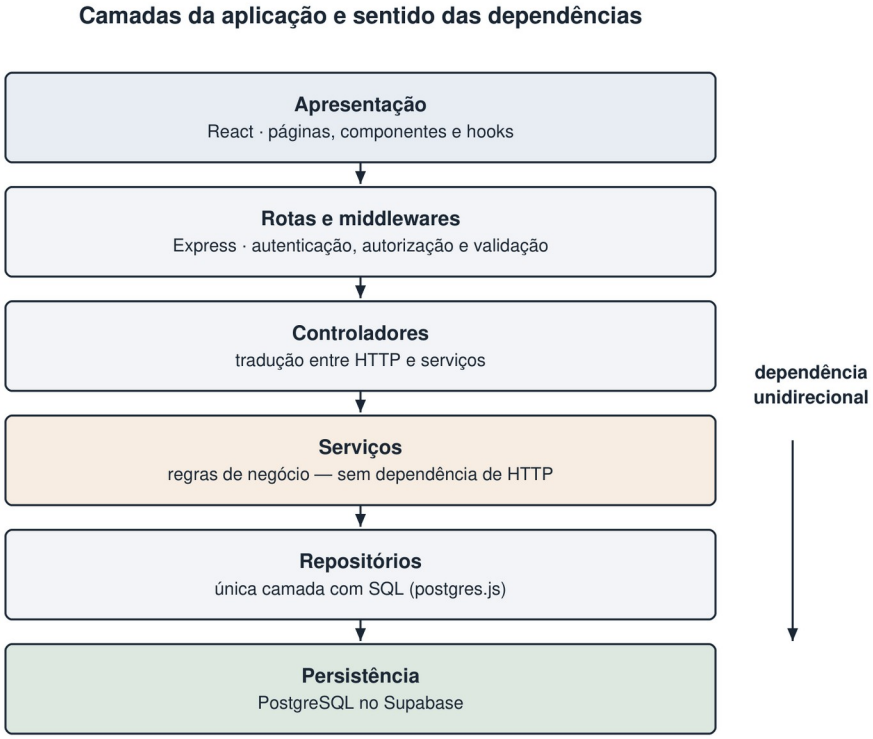


Fonte: Elaborado pelos autores (2026).

4.2 Arquitetura em camadas

O servidor adota arquitetura em camadas com dependência unidirecional: cada camada conhece apenas a imediatamente inferior. A Figura 3 apresenta essa organização.

Figura 3 — Camadas da aplicação e sentido das dependências



Fonte: Elaborado pelos autores (2026).

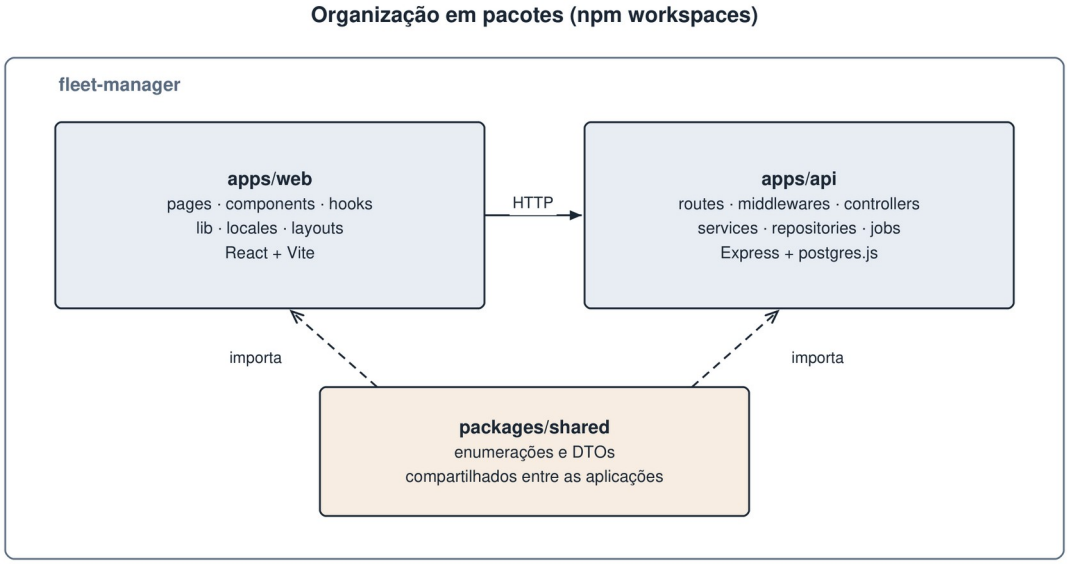
A responsabilidade de cada camada é delimitada de forma estrita. As rotas declaram os caminhos, definem o esquema de validação e encadeiam os *middlewares*, sem conter lógica. Os controladores traduzem a requisição HTTP em chamada de serviço e o retorno em resposta HTTP, sem conter regra de negócio. Os serviços concentram as regras de negócio e não conhecem o Express: não recebem os objetos de requisição e resposta. Os repositórios encapsulam todo o acesso ao banco e constituem a única camada em que há SQL.

O isolamento da camada de serviços em relação ao protocolo HTTP é o que viabiliza os testes automatizados descritos na seção 5.2: os serviços são exercitados com repositórios substituídos por dublês, sem necessidade de servidor ou banco em execução.

4.3 Organização do repositório

O código é organizado em *monorepo*, conforme a Figura 4. A escolha justifica-se pelo compartilhamento de tipos entre a interface e o servidor: uma alteração em uma enumeração provoca erro de compilação em ambos os lados, o que impede a divergência silenciosa do contrato de dados.

Figura 4 — Organização em pacotes do *monorepo*



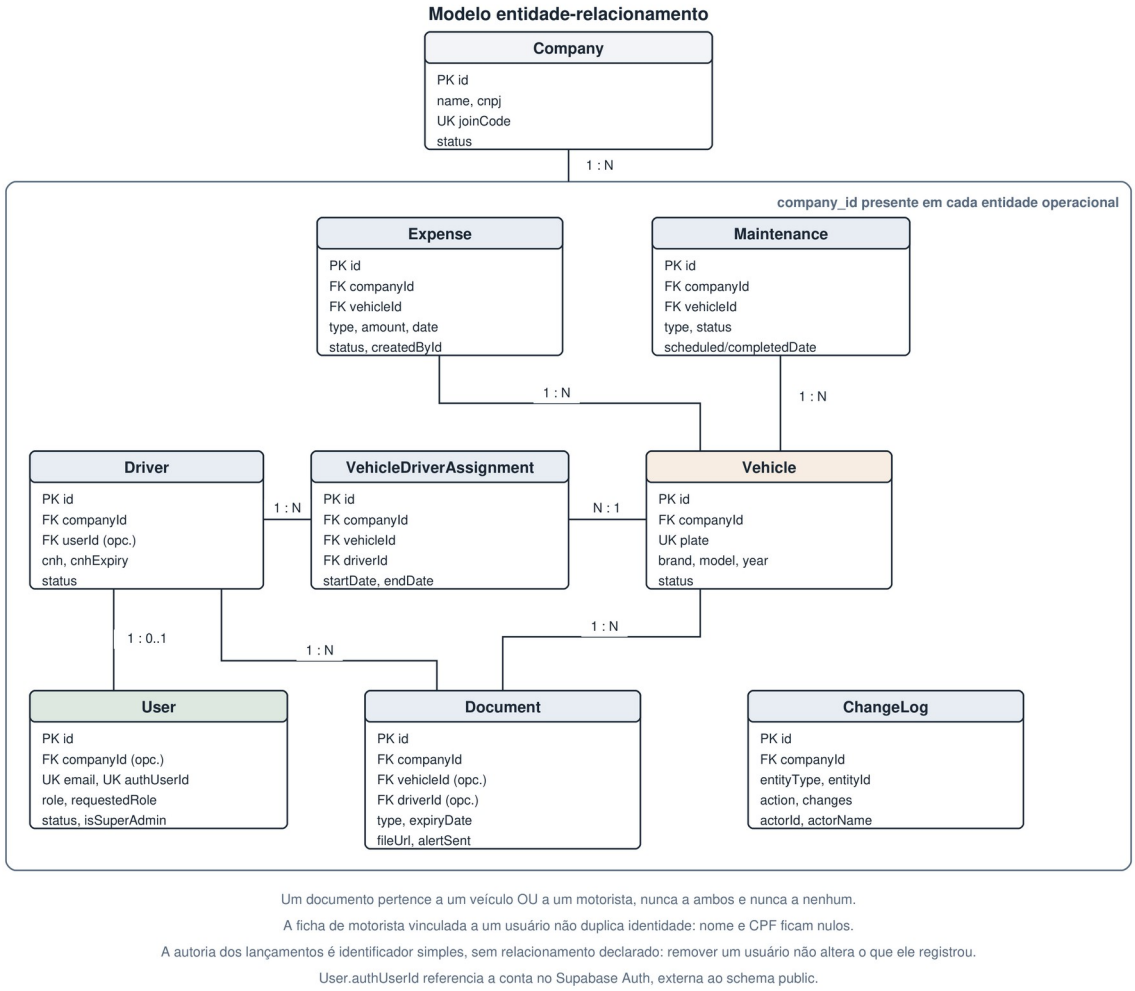
Fonte: Elaborado pelos autores (2026).

O pacote compartilhado publica suas definições em dois formatos. A exigência decorre dos consumidores: o ambiente de execução do servidor requer módulos CommonJS, ao passo que o empacotador da interface exige módulos ECMAScript para resolver importações nomeadas. A compilação ocorre automaticamente após a instalação das dependências.

4.4 Modelo de dados

O modelo é composto por nove entidades, organizadas em torno de duas fronteiras: a entidade Empresa, que delimita o alcance de todo dado operacional, e a entidade Veículo, que concentra as três dimensões da gestão de frota. A Figura 5 apresenta o modelo entidade-relacionamento, e o Quadro 3 descreve a finalidade de cada entidade.

Figura 5 — Modelo entidade-relacionamento



Fonte: Elaborado pelos autores (2026).

Quadro 3 — Entidades do modelo de dados

Entidade	Finalidade e observações
Company	Organização cliente. Fronteira de isolamento: seu identificador participa da condição de toda consulta operacional. Possui código de adesão próprio, apresentado no cadastro.
User	Perfil de acesso: dados cadastrais, papel efetivo, papel solicitado e situação de aprovação. As credenciais residem no serviço de autenticação.
Vehicle	Veículo da frota. Entidade central da operação; a placa é única dentro da empresa.
Driver	Ficha de motorista, com número e validade da habilitação. Quando vinculada a um usuário, nome e CPF ficam nulos: a identidade pertence ao perfil, e não é duplicada.
VehicleDriverAssignment	Vínculo entre veículo e motorista, com data de início e de término. Encerrar não apaga: o período permanece consultável.
Expense	Despesa operacional, obrigatoriamente vinculada a um veículo. O valor usa tipo decimal, evitando erro de arredondamento. Pode ser cancelada com motivo, sem deixar de existir.

Entidade	Finalidade e observações
Maintenance	Manutenção preventiva ou corretiva, com data prevista e de conclusão. A situação "atrasada" não é gravada: decorre da data prevista.
Document	Documento obrigatório com data de vencimento e anexo digital opcional. Vincula-se a um veículo ou a um motorista. Registra se e quando a rotina diária o sinalizou.
ChangeLog	Histórico de alterações: entidade afetada, ação, campos alterados, autor e motivo. Grava o nome do autor por cópia.

Fonte: Elaborado pelos autores (2026).

Cinco aspectos do modelo merecem registro.

Primeiro, o identificador da empresa está presente em cada entidade operacional, e não é derivado percorrendo relacionamentos. A decisão é deliberada: obter a empresa por caminho indireto tornaria o isolamento dependente da correção de cada junção, ao passo que a coluna própria permite que toda consulta a declare diretamente na condição.

Segundo, a obrigatoriedade do vínculo entre despesa e veículo, garantida por restrição no banco, é o que torna confiável a apuração do custo por ativo.

Terceiro, nenhuma situação de prazo é armazenada. A de vencimento do documento e a da habilitação são calculadas a cada consulta a partir da data de validade; a manutenção programada cuja data prevista ficou para trás é devolvida como atrasada. Como ninguém precisa lembrar de atualizar uma situação, o dado não fica defasado. As datas de vencimento empregam tipo de data civil, sem hora nem fuso horário, de modo que o dia exibido não dependa da localização de quem consulta.

Quarto, a relação entre veículo e motorista é de muitos para muitos com período, representada por entidade própria. Substituição e compartilhamento são operação corriqueira em uma frota, e registrar apenas o vínculo vigente impediria responder quem conduzia determinado veículo em uma data passada.

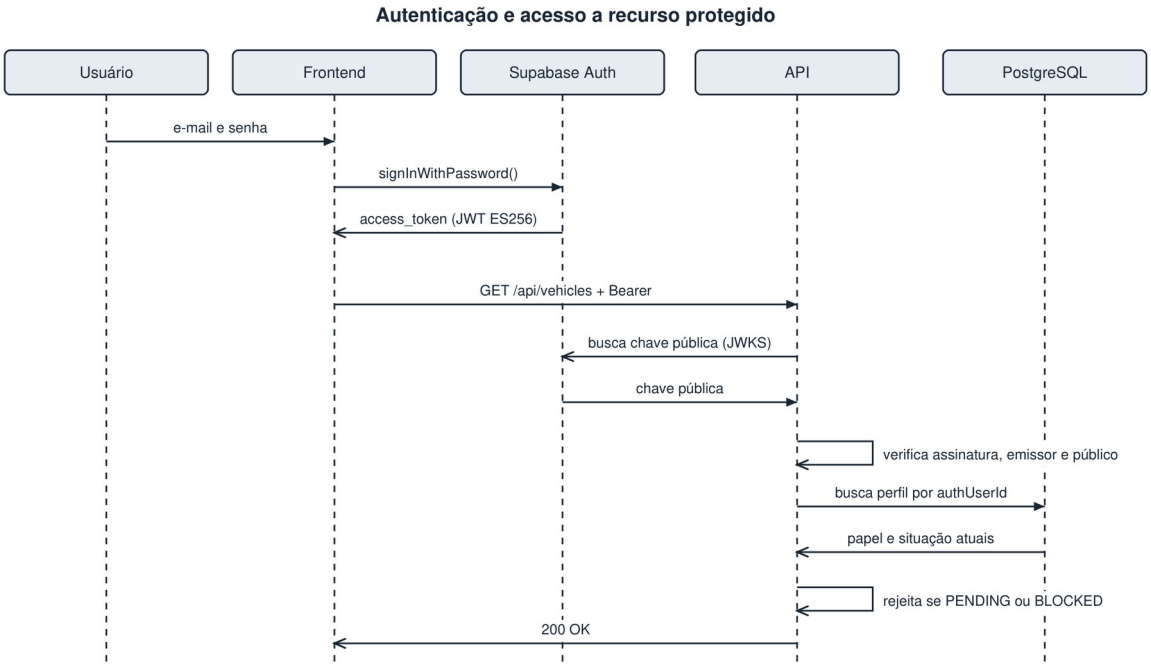
Quinto, a autoria dos lançamentos não possui relacionamento declarado com a entidade de usuário: é um identificador simples. Assim, remover um usuário não altera nem apaga os lançamentos que ele registrou. O nome legível do autor é obtido do histórico de alterações, que o grava por cópia no momento da ação.

Registra-se, ainda, que a ficha de motorista e o perfil de usuário passaram a ser vinculáveis: aprovar alguém como operador cria a ficha correspondente, de forma idempotente. As colunas de nome e documento na ficha permanecem no modelo para as fichas anteriores à integração, que não possuem usuário associado.

4.5 Autenticação e controle de acesso

A Figura 6 apresenta o fluxo de autenticação e o acesso subsequente a um recurso protegido.

Figura 6 — Autenticação e acesso a recurso protegido

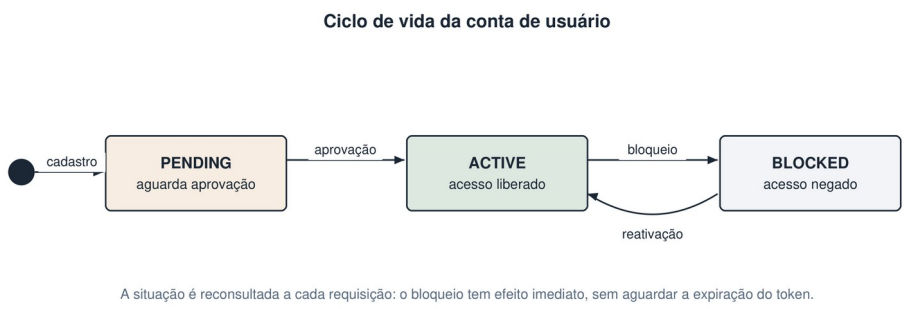


Fonte: Elaborado pelos autores (2026).

O cadastro ocorre em duas etapas. A interface cria a conta no serviço de autenticação e obtém a sessão; de posse do *token*, envia os dados cadastrais à aplicação, que cria o perfil com situação pendente. Quando já existe perfil com o mesmo endereço de correio eletrônico e sem conta vinculada, a aplicação associa o perfil existente em vez de criar outro, preservando papel e situação. Caso a conta esteja autenticada mas ainda não possua perfil, a aplicação responde com código específico e a interface conduz o usuário à conclusão do cadastro.

A Figura 7 apresenta o ciclo de vida da conta. Contas recém-criadas permanecem pendentes até que um administrador as aprove, e o bloqueio produz efeito imediato, pois o perfil é reconsultado a cada requisição.

Figura 7 — Ciclo de vida da conta de usuário



Fonte: Elaborado pelos autores (2026).

O controle de acesso opera dentro de uma organização. Cada perfil pertence a uma empresa, e o identificador dessa empresa participa da condição de toda consulta e de toda escrita. Esse identificador é obtido do perfil reconsultado no banco a cada requisição, e nunca de dado enviado pelo cliente — inclusive o *token*, que poderia carregá-lo, não é a fonte. O ingresso em uma organização ocorre por código de adesão informado no cadastro.

Acima das organizações existe um perfil de administração da plataforma, que não pertence a nenhuma delas e escolhe, a cada sessão, em qual vai operar. Duas decisões sustentam essa condição. A primeira é que ela não constitui um papel: reside em coluna própria da entidade de usuário, e não na enumeração de papéis. O papel é dado de entrada em dois pontos — o cadastro registra o solicitado, a aprovação registra o efetivo — e um valor adicional na enumeração tornaria possível pleitear ou conceder a condição por esses caminhos. A segunda é que a empresa escolhida trafega em cabeçalho próprio, e é aceita somente após o servidor confirmar, no banco, que o perfil autenticado possui aquela condição; para qualquer outro perfil o cabeçalho é ignorado.

O recorte do operador é mais estreito que o da organização. Ele alcança os lançamentos de sua própria autoria, os veículos a que está vinculado no momento e os próprios documentos pessoais. Compartilhar um veículo com outro motorista não concede acesso aos lançamentos nem aos documentos pessoais dele. A verificação é do servidor: restringir apenas o que a interface oferece deixaria a regra a cargo da tela, e uma requisição construída manualmente a contornaria.

O Quadro 4 apresenta a matriz de controle de acesso efetivamente aplicada, extraída da configuração das rotas e dos serviços. Onde o operador tem acesso parcial, a célula indica o recorte.

Quadro 4 — Matriz de controle de acesso por papel

Recurso ou operação	Administrador	Gestor	Operador
Aprovar, recusar e bloquear contas; alterar papéis	Sim	Não	Não
Listar os usuários da empresa	Sim	Sim	Não
Editar o próprio perfil	Sim	Sim	Sim
Consultar veículos e motoristas	Empresa	Empresa	Só os vinculados a si
Cadastrar e editar veículos, motoristas e documentos	Sim	Sim	Não
Abrir e encerrar vínculos entre motorista e veículo	Sim	Sim	Não
Registrar despesas e manutenções	Empresa	Empresa	Só em veículo vinculado
Consultar lançamentos, painel e central de alertas	Empresa	Empresa	Só os próprios
Corrigir e cancelar lançamento, com motivo	Qualquer	Qualquer	Só os próprios
Excluir lançamento em definitivo	Sim	Sim	Não
Desativar veículo ou motorista	Sim	Sim	Não
Excluir veículo ou motorista em definitivo	Sim	Não	Não

Fonte: Elaborado pelos autores (2026).

Duas proteções completam a matriz. Ninguém altera a situação da própria conta nem a exclui. E a empresa nunca fica sem administrador ativo: rebaixar, bloquear ou excluir o último deles é recusado, pois só o administrador aprova contas e altera papéis, e sem ele a empresa ficaria impedida de administrar o próprio acesso. A verificação trava as linhas dos administradores ativos durante a alteração, para que dois administradores que se rebaixem ao mesmo tempo não deixem a empresa sem nenhum. A regra substituiu uma proteção anterior, que valia apenas para uma conta de demonstração identificada pelo endereço de correio eletrônico.

O perfil de administração da plataforma não aparece na matriz porque não é um papel: ao escolher uma empresa, ele opera dentro dela como administrador, e fora dela apenas cadastra empresas e altera a situação delas.

4.6 Módulos funcionais e interface

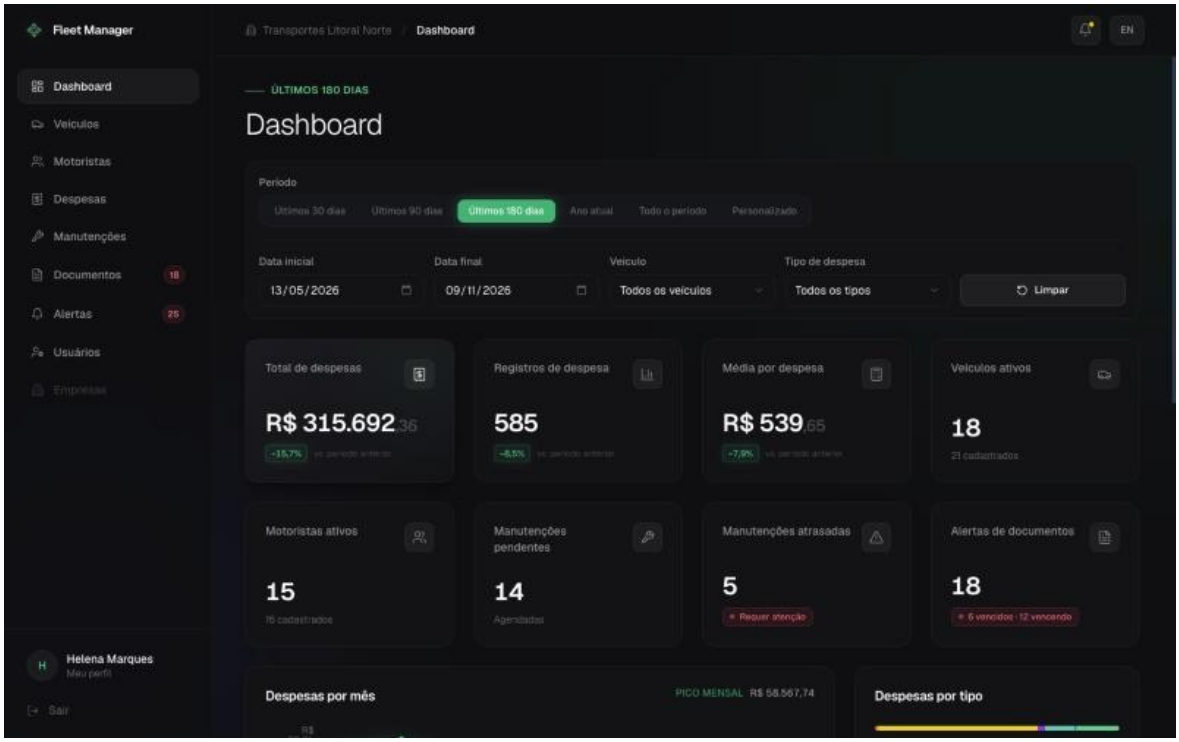
Os módulos do sistema são descritos a seguir. O de veículos oferece cadastro, edição, consulta com filtros, desativação, reativação e exclusão permanente. O de motoristas segue estrutura equivalente e acrescenta a validade da habilitação. O vínculo entre ambos é de

muitos para muitos, com tela dedicada; um motorista com habilitação vencida não pode ser vinculado a um veículo novo.

O módulo de despesas registra lançamentos obrigatoriamente vinculados a um veículo, distribuídos em seis categorias — combustível, manutenção, multa, IPVA, seguro e outras —, com filtros por veículo, categoria e período, e não aceita data posterior ao dia corrente. O de manutenções distingue as preventivas das corretivas e acompanha quatro situações: programada, realizada, atrasada e cancelada. O de documentos registra a data de vencimento, classifica automaticamente a situação e permite anexar o arquivo digital. Nos três, um lançamento errado é cancelado com motivo, sem ser apagado: permanece na lista, marcado, sai dos totais e pode ser reativado, e cada alteração fica no histórico com autor, data, campos alterados e motivo, consultável pela opção Histórico de cada linha de despesas e manutenções. As listagens de despesas e manutenções são paginadas, cinquenta registros por vez, com o total informado no cabeçalho X-Total-Count da resposta.

O painel de indicadores, apresentado na Figura 8, apura custo total, custo médio, quantidade de lançamentos, evolução mensal, distribuição por categoria, os veículos de maior custo e os lançamentos mais recentes, com filtros por período, veículo e categoria. Cada indicador financeiro vem acompanhado da variação em relação ao período imediatamente anterior, de mesma duração, o que mostra se o custo subiu ou caiu sem exigir outra consulta. Todas as somas são feitas no banco, que devolve apenas os totais.

Figura 8 — Painel de indicadores com um ano de operação fictícia

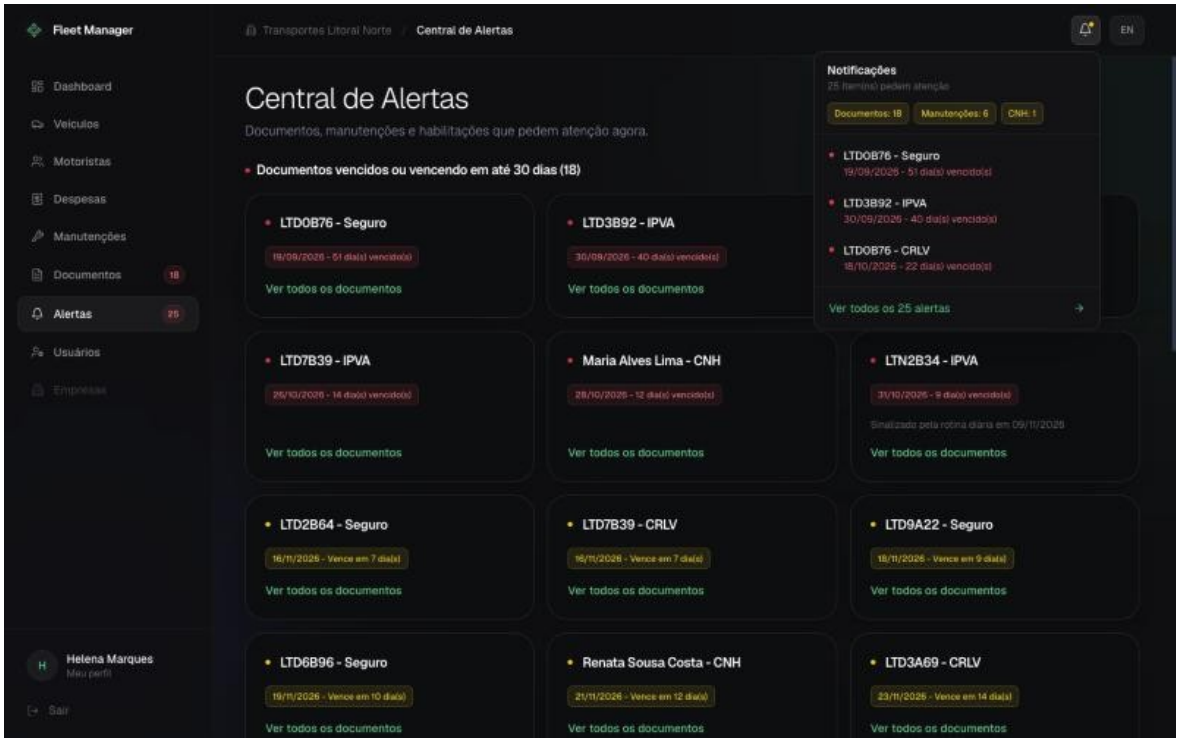


Fonte: Elaborado pelos autores (2026).

A central de alertas, apresentada na Figura 9, responde à pergunta "o que precisa ser resolvido agora" em três frentes: documentos vencidos ou que vencem em até trinta dias; manutenções atrasadas ou previstas para os próximos sete dias; e habilitações vencidas ou que vencem em até trinta dias. A janela de manutenção é menor porque a data é escolhida pela própria empresa, com antecedência curta; a de documentos e habilitações é maior porque a renovação depende de terceiros. Quando a mesma habilitação existe como documento anexado e como validade na ficha do motorista, ela aparece uma única vez. O menu lateral exibe o total de alertas e o de documentos.

O sino do cabeçalho abre, por cima da tela em que se está, um painel de notificações com a contagem de cada frente e os três alertas mais urgentes — primeiro o que já venceu ou atrasou, depois o prazo mais próximo. Cada item leva à tela correspondente, e o rodapé leva à central completa. São duas visões do mesmo dado: o painel responde ao que é mais urgente sem tirar o usuário do que estava fazendo; a central mostra tudo.

Figura 9 — Central de alertas, com o painel de notificações do cabeçalho aberto



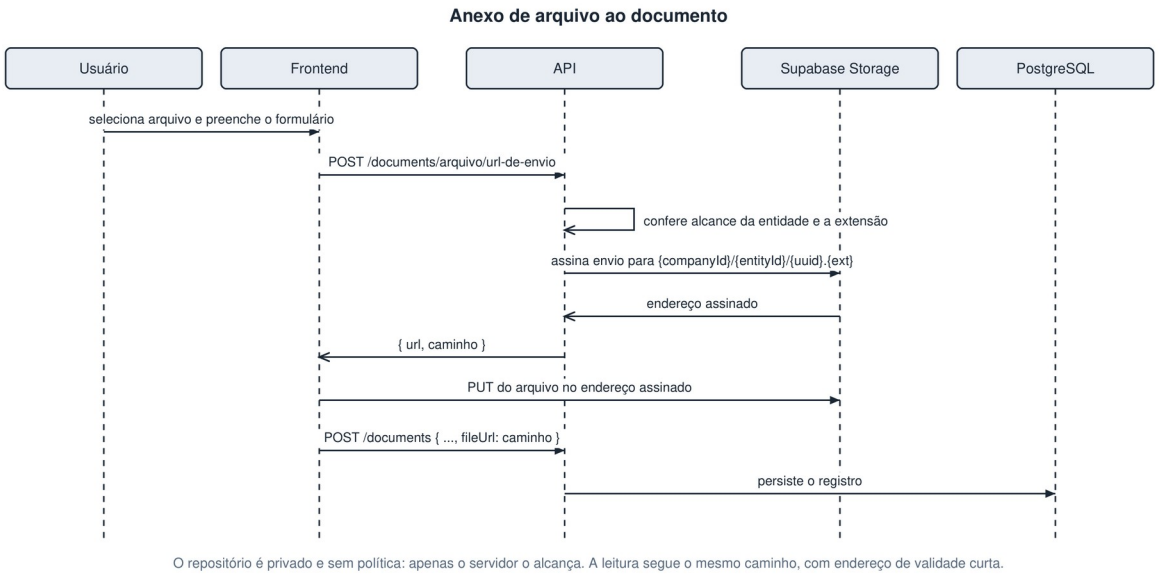
Fonte: Elaborado pelos autores (2026).

A administração de usuários permite aprovar e recusar solicitações, alterar papel e situação e excluir contas; o perfil de administração da plataforma dispõe ainda de tela para cadastrar empresas. A interface segue o *design system* Ledger, de fundo escuro e um único destaque de cor, e está disponível em português brasileiro e em inglês. As listas de seleção dos filtros e formulários são componentes do próprio sistema, e não as do navegador: a lista nativa é desenhada pelo sistema operacional e destoava da interface. O componente mantém o uso por teclado — setas, Enter, Esc e a busca pela primeira letra — e a validação de campo obrigatório. Trocar de tela pelo menu é imediato; a animação de entrada fica restrita à primeira tela depois do login.

4.7 Armazenamento de arquivos

O anexo de arquivos aos documentos utiliza o serviço de armazenamento do Supabase. O repositório é privado e não possui política de acesso alguma, de modo que credenciais de navegador não o alcançam: apenas o servidor o acessa, com a chave de serviço. A Figura 10 apresenta o fluxo.

Figura 10 — Anexo de arquivo ao documento



Fonte: Elaborado pelos autores (2026).

Tanto a leitura quanto o envio passam pela aplicação, que emite endereços assinados de validade curta. Para ler, o cliente informa o identificador do documento, não o caminho do arquivo; a aplicação recupera o documento pelo mesmo procedimento usado na consulta comum e, se ele não estiver ao alcance de quem pede, responde como se não existisse — sem chegar a assinar endereço algum. Para enviar, a aplicação decide o caminho de destino e devolve um endereço de gravação; o caminho começa pelo identificador da empresa, obtido do perfil autenticado.

A decisão é consequência de uma verificação. A alternativa, adotada em versão anterior, era o envio direto do navegador ao serviço de armazenamento, com o controle de acesso expresso nas políticas do repositório. Esse desenho reduz a carga sobre o servidor, mas obriga a reescrever a regra de autorização na linguagem do armazenamento — e a regra da aplicação é mais estreita do que a pertinência à mesma organização: um operador alcança os próprios documentos pessoais e os dos veículos a que está vinculado no momento, não o documento pessoal de um colega. Uma política capaz de distinguir isso duplicaria, em outra linguagem, lógica que a camada de serviços já aplica; duas expressões da mesma regra divergem com o tempo. Concentrar a decisão em um único ponto elimina a duplicação, ao custo de uma requisição adicional para obter o endereço assinado.

Três consequências acompanham o desenho. A validação do tipo de arquivo passa a ser responsabilidade da aplicação, que aceita apenas imagens nos formatos correntes e documentos em PDF. A substituição de um anexo e a exclusão de um documento removem o arquivo correspondente do repositório, operação realizada após a confirmação da transação e

cuja eventual falha é registrada sem interromper a resposta. E o endereço de leitura expira, de modo que sua divulgação acidental não concede acesso permanente.

4.8 Rotina automatizada de vencimentos

Uma rotina diária identifica os documentos que entram na janela de trinta dias e registra a data em que os sinalizou, de forma idempotente: documentos já sinalizados não são reprocessados, e execuções repetidas no mesmo dia não produzem efeito colateral. A central de alertas exibe essa data em cada documento, o que torna visível que o prazo foi detectado pelo sistema, e não por alguém que abriu a tela. Renovar o documento — gravar uma nova data de vencimento — limpa a sinalização, para que a rotina volte a avisar quando o novo prazo se aproximar.

Delimita-se explicitamente o alcance dessa rotina: ela não envia notificação para fora do sistema. O acompanhamento de prazos é realizado dentro da aplicação, pela central de alertas e pelos indicadores de vencimento. A notificação por canais externos foi avaliada e excluída do escopo, conforme a seção 4.13. A situação exibida na central não depende da rotina: ela é calculada a cada consulta, e a rotina apenas registra quando o documento entrou na janela.

4.9 Proteção do banco de dados

A autorização é aplicada na camada de aplicação, e o banco é acessado exclusivamente pelo servidor. Ocorre, porém, que a plataforma de hospedagem expõe automaticamente o esquema público por uma interface REST gerada, acessível com a chave pública do projeto — chave que, por natureza, é distribuída junto com a interface. Sem proteção, as tabelas seriam legíveis e graváveis por quem a obtivesse, contornando integralmente a aplicação e o controle de acesso por papel.

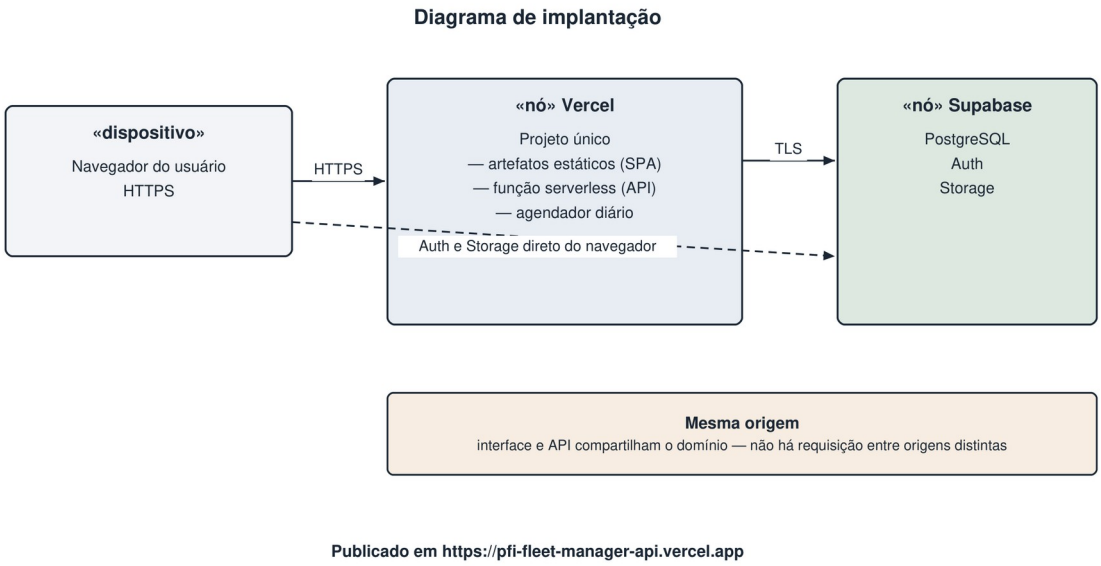
Adotaram-se, portanto, duas camadas de proteção. A primeira habilita a segurança em nível de linha em todas as tabelas, sem criar política alguma, o que nega acesso aos papéis públicos; o papel proprietário das tabelas não é submetido a essa restrição, de modo que a aplicação opera normalmente. A segunda revoga os privilégios desses papéis sobre o esquema, inclusive para tabelas criadas futuramente. A ausência de políticas é intencional: elas só fariam sentido se o cliente consultasse o banco diretamente, o que esta arquitetura não

faz. O objetivo é fechar um caminho de acesso que a plataforma abre por padrão, e não replicar a autorização no banco.

4.10 Publicação em produção

O sistema é publicado como projeto único, com a interface estática e a camada de serviço atendidas pelo mesmo domínio, conforme a Figura 11.

Figura 11 — Diagrama de implantação



Fonte: Elaborado pelos autores (2026).

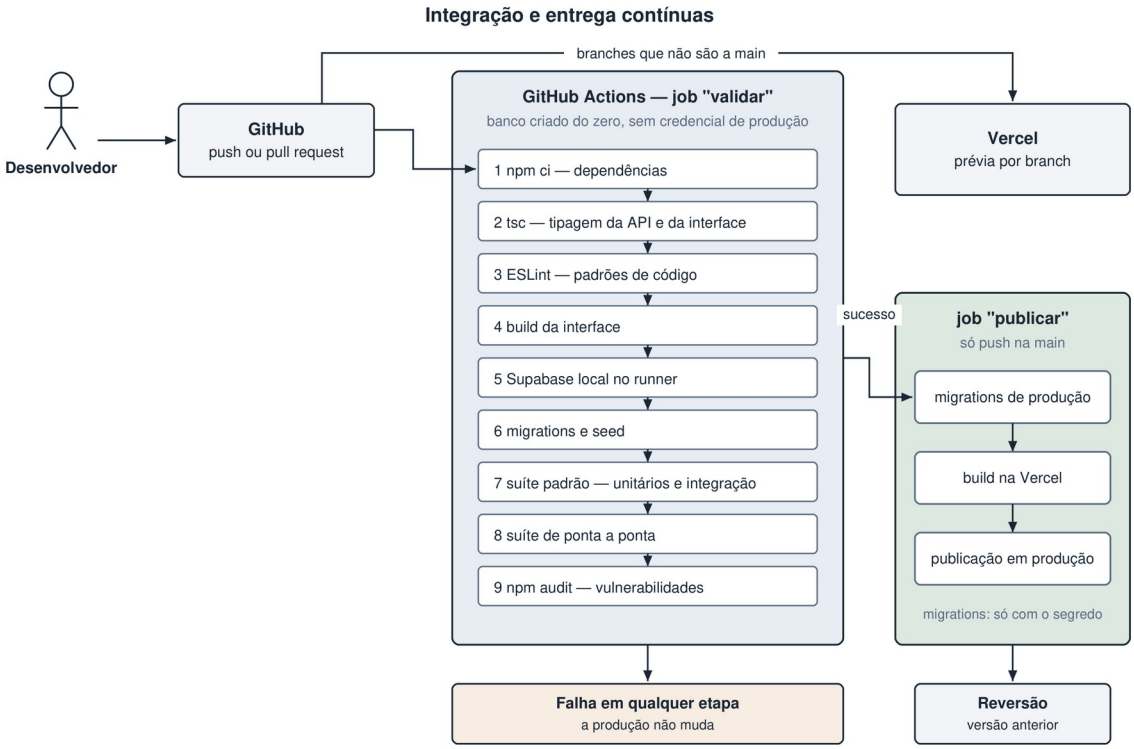
A decisão pelo domínio compartilhado tem consequência direta: não há requisição entre origens distintas entre a interface e a camada de serviço, o que elimina toda uma classe de configuração e de falhas. A plataforma inicialmente prevista para o servidor foi substituída após verificar-se que não oferece mais camada gratuita permanente, condição necessária à continuidade do projeto até a apresentação.

A adaptação ao modelo de execução sem servidor dedicado impôs dois ajustes ao código. O primeiro decorre da ausência de processo persistente entre requisições: o agendamento interno da rotina de vencimentos é desativado em produção, e a execução passa a ser disparada pelo agendador da plataforma, por meio de rota protegida por segredo. O segundo refere-se ao pacote compartilhado, cuja publicação em código-fonte TypeScript não é carregável pelo ambiente de execução, o que motivou a compilação descrita na seção 4.3.

4.11 Integração e entrega contínuas

A Figura 12 apresenta o fluxo que se segue a cada alteração enviada ao repositório.

Figura 12 — Integração e entrega contínuas



Fonte: Elaborado pelos autores (2026).

A integração contínua roda no GitHub Actions a cada *pull request* e a cada envio à ramificação principal, em nove etapas executadas em ordem: instalação das dependências, verificação de tipos da interface de programação e da interface, análise estática, construção da interface, criação de um Supabase local dentro da própria máquina de execução, aplicação das *migrations* e da carga de dados fictícios, suíte padrão, suíte de ponta a ponta e auditoria de dependências. O banco é criado do zero e descartado ao fim, e nenhuma credencial de produção é usada. A execução completa leva cerca de dois minutos.

A entrega contínua é o segundo trabalho do mesmo fluxo, chamado publicar. Ele existe apenas nos envios à ramificação principal e só começa depois que a validação termina com sucesso: se qualquer etapa falhar, a produção não muda. A publicação automática da Vercel na ramificação principal foi desligada na configuração do projeto, de modo que esse trabalho é o único caminho até a produção. Ele aplica as *migrations* pendentes ao banco de produção, quando a credencial correspondente está cadastrada, e só então constrói e publica a versão — a ordem garante que o código nunca chegue antes da estrutura de que depende. As

demais ramificações continuam recebendo da Vercel uma prévia com endereço próprio, sem tocar na produção, e qualquer versão anterior pode voltar ao ar sem nova construção.

A versão anterior do fluxo publicava a cada envio, em paralelo com os testes, e um teste reprovado não impedia a publicação. A correção foi identificada na revisão descrita na seção 3.1.

4.12 Observabilidade e tratamento de falhas

Observabilidade é a capacidade de saber, de fora, se o sistema está no ar, onde falha e quão rápido responde. O endereço /health consulta o banco com prazo de três segundos e responde 200 quando ele atende e 503 quando não, o que permite a um monitor externo detectar a queda. Cada requisição atendida gera uma linha de registro com método, caminho, situação e duração. Uma requisição que ultrapassa dez segundos recebe 503 e fica registrada, em vez de prender a conexão. Na produção, a plataforma agrupa os erros por tipo e mantém um *firewall* à frente da aplicação. A falha de uma consulta ao banco é tratada na própria promessa, de modo que uma resposta tardia não derrube o processo.

4.13 Delimitação do escopo

Não integram o escopo deste trabalho o rastreamento por satélite em tempo real, a telemetria avançada, o planejamento e a otimização de rotas, a integração automática com sistemas governamentais e o desenvolvimento de aplicativo móvel nativo. A solução é entregue como aplicação web responsiva, acessível por navegador em dispositivos móveis.

Excluiu-se igualmente a notificação de vencimentos por correio eletrônico, mensagem de texto ou canal equivalente. Tal recurso exigiria a contratação e a configuração de serviço externo de envio, com custo e dependência operacional não justificáveis no contexto do trabalho. O acompanhamento de prazos é realizado dentro da aplicação, o que atende ao propósito de substituir o controle passivo por acompanhamento ativo sem introduzir dependência de infraestrutura de terceiros. Como consequência dessa exclusão, não se implementou a recuperação autônoma de senha, que depende do mesmo canal; a alteração de senha permanece disponível ao usuário autenticado.

5 RESULTADOS E DISCUSSÕES

Este capítulo apresenta o que foi efetivamente entregue, a verificação realizada sobre o sistema e as limitações identificadas. Os resultados de teste vêm de duas rodadas: a de 20 de outubro de 2026, documentada no Relatório de Testes do projeto, e a de 9 de novembro de 2026, executada após a revisão das regras de negócio. Cada tabela indica a rodada de origem.

5.1 Requisitos e regras de negócio

Os dez requisitos funcionais definidos no termo de abertura foram atendidos. O Quadro 5 relaciona cada requisito à sua situação.

Quadro 5 — Requisitos funcionais e situação de atendimento

Código	Requisito	Situação
RF01	Cadastro, edição e exclusão de veículos	Atendido
RF02	Cadastro e gerenciamento de motoristas	Atendido
RF03	Despesas vinculadas obrigatoriamente a um veículo	Atendido
RF04	Registro de diferentes categorias de despesa	Atendido
RF05	Controle de manutenções preventivas e corretivas	Atendido
RF06	Registro de documentos com data de vencimento	Atendido
RF07	Emissão de alertas de vencimento	Atendido dentro da aplicação
RF08	Indicadores financeiros por veículo e por período	Atendido
RF09	Autenticação de usuários	Atendido
RF10	Controle de acesso por papel	Atendido

Fonte: Elaborado pelos autores (2026).

O RF07 merece esclarecimento quanto ao canal de emissão. O sistema classifica automaticamente documentos, manutenções e habilitações por situação de prazo, consolida-os na central de alertas, exibe contadores no menu e executa diariamente a rotina de sinalização. A notificação por canais externos, contudo, foi excluída do escopo, conforme a seção 4.13. O requisito é, portanto, atendido no sentido da exibição, e não no da notificação ativa fora do sistema — delimitação declarada de forma explícita para que o alcance não seja interpretado de maneira ampliada.

Além dos requisitos previstos no levantamento inicial, o desenvolvimento incorporou funcionalidades identificadas durante o trabalho: segregação de dados por organização, com ingresso por código de adesão e perfil de administração da plataforma acima das

organizações; vínculo de muitos para muitos entre veículos e motoristas, com registro do período de vigência; anexo de arquivo digital aos documentos, servido pela aplicação com endereço assinado de validade curta; cancelamento de lançamentos com motivo, sem apagá-los; histórico de alterações com autor, campos modificados e motivo; fluxo de aprovação de contas por administrador; bloqueio e reativação de contas; edição do perfil próprio; distinção entre desativação e exclusão permanente; listagens paginadas; e interface em português brasileiro e inglês.

O Quadro 6 apresenta a situação de atendimento dos requisitos não funcionais.

Quadro 6 — Requisitos não funcionais e situação de atendimento

Código	Requisito	Situação
RNF01	Aplicação web responsiva	Atendido
RNF02	Armazenamento seguro em banco relacional	Atendido
RNF03	Controle de autenticação e autorização	Atendido
RNF04	Tempo de resposta adequado nas operações comuns	Atendido, com medição (seção 5.4)
RNF05	Arquitetura cliente-servidor	Atendido
RNF06	Escalabilidade para novas funcionalidades	Atendido
RNF07	Integridade e consistência das informações	Atendido

Fonte: Elaborado pelos autores (2026).

O RNF04 não tinha meta numérica no levantamento inicial. Adotou-se, nesta revisão, o limite de um segundo de Nielsen (1993), aplicado ao percentil 99 das respostas da interface de programação: 99 de cada 100 respostas devem chegar em menos de um segundo. A seção 5.4 mostra que o limite foi atendido em todos os cenários medidos.

O Quadro 7 relaciona as regras de negócio implementadas. Para cada uma, registra-se o motivo pelo qual existe — o problema que ela evita para quem usa o sistema. A revisão descrita na seção 3.1 manteve as regras cujo benefício é claro e proporcional ao esforço que exigem do usuário, substituiu a proteção de uma conta fixa pela RN22 e acrescentou a RN14, a RN15 e a RN22.

Quadro 7 — Regras de negócio implementadas e motivo de cada uma

Código	Regra	Por que existe
RN01	Toda despesa pertence a um veículo.	Sem isso, o custo por veículo, que é a pergunta central do gestor, não fecha.
RN02	Toda manutenção pertence a um veículo.	O histórico de manutenção é o prontuário do veículo e apoia a decisão de revisar, manter ou vender.

Código	Regra	Por que existe
RN03	Um documento pertence a um veículo ou a um motorista, nunca a ambos e nunca a nenhum.	CRLV e seguro são do veículo; CNH é da pessoa. Misturar impediria saber quem deve renovar.
RN04	Conta nova fica pendente até um administrador aprovar; o papel pedido no cadastro não vira permissão sozinho.	Qualquer pessoa pode criar conta no serviço de autenticação; quem entra na frota é decisão da empresa.
RN05	Conta bloqueada perde o acesso na requisição seguinte, mesmo com *token* válido.	Desligamentos têm efeito imediato, sem esperar o *token* expirar.
RN06	Placa, CPF e CNH do motorista são únicos dentro da empresa; CPF e endereço de correio eletrônico da conta, em todo o sistema.	Evita o mesmo veículo ou a mesma pessoa em duas fichas, com o histórico dividido entre elas.
RN07	A exclusão definitiva de veículo ou motorista é do administrador, exige desativação prévia, mostra antes o que será apagado e remove em cascata os registros dependentes.	Ação sem volta pede duas etapas e quem responde pela empresa.
RN08	Desativar preserva o registro e o histórico e encerra os vínculos vigentes do veículo.	O veículo vendido sai da operação, mas o custo que teve continua nos indicadores do período.
RN09	Concluir a manutenção grava a data de conclusão; reabri-la a apaga.	A data de conclusão é o registro de que o serviço foi feito.
RN10	As situações de prazo — documento e habilitação vencidos ou a vencer, manutenção atrasada — são calculadas a cada consulta, nunca gravadas.	Ninguém precisa lembrar de atualizar uma situação; o dado não fica defasado.
RN11	A manutenção programada cuja data prevista passou é apresentada como atrasada; essa situação não pode ser escolhida manualmente.	O atraso aparece sem depender de alguém marcá-lo.
RN12	A central de alertas reúne documentos vencidos ou que vencem em até 30 dias, manutenções atrasadas ou previstas para os próximos 7 dias e CNH vencida ou que vence em até 30 dias, sem repetir a mesma CNH.	Uma única tela responde ao que precisa ser resolvido agora.
RN13	A rotina diária registra quando sinalizou cada documento na janela de 30 dias; renovar o documento limpa a sinalização.	Mostra que o prazo foi detectado pelo sistema e reinicia o ciclo para o novo vencimento.
RN14	Motorista com CNH vencida não pode ser vinculado a um veículo novo; o vínculo existente não é desfeito.	Evita entregar veículo a quem não pode dirigir. Desfazer vínculos sozinho tiraria do gestor uma decisão que é dele.
RN15	Despesa não pode ter data posterior ao dia corrente.	Despesa é gasto realizado; gasto previsto é manutenção programada. Evita indicadores com custo que ainda não existe.

Código	Regra	Por que existe
RN16	Cancelar um lançamento exige motivo; ele permanece visível, sai dos totais e pode ser reativado. A exclusão definitiva é do gestor e do administrador.	Corrige o erro sem apagar o rastro de quem lançou, quem cancelou e por quê.
RN17	Toda criação, alteração, cancelamento e exclusão grava autor, campos alterados e motivo, quando houver.	Responde a "quem mudou isto?" sem depender de memória.
RN18	Todo dado operacional pertence a uma empresa, identificada pelo perfil consultado no banco e nunca por dado enviado pelo cliente.	É o que permite atender várias frotas na mesma base sem que uma veja a outra.
RN19	O operador alcança os lançamentos que fez, os veículos a que está vinculado agora e os próprios documentos; compartilhar veículo não dá acesso aos dados do colega.	Privacidade entre colegas e menor chance de alterar o que não é seu.
RN20	O operador só registra despesa ou manutenção em veículo a que está vinculado.	Evita lançar custo no veículo errado.
RN21	O arquivo anexado segue o alcance do documento; o endereço de leitura vale sessenta segundos.	Um endereço divulgado por engano não vira acesso permanente.
RN22	A empresa nunca fica sem administrador ativo.	Sem administrador, ninguém aprova contas nem corrige papéis.
RN23	Os dados de identificação do veículo são gravados em maiúsculas.	A mesma placa digitada de dois jeitos continua sendo um só veículo.

Fonte: Elaborado pelos autores (2026).

5.2 Verificação funcional

O Quadro 8 resume as verificações executadas sobre o código em 9 de novembro de 2026.

Quadro 8 — Verificações executadas sobre o código

Verificação	Resultado
Compilação TypeScript do servidor e da interface	Sem erros
Análise estática com ESLint	Sem erros nem avisos
Testes unitários	213 aprovados
Testes de integração com o banco	52 aprovados
Testes de ponta a ponta	137 aprovados
Construção do pacote de produção	Concluída
Auditoria de dependências	Nenhuma vulnerabilidade conhecida
Integração contínua	Nove etapas, cerca de dois minutos; a publicação só ocorre depois delas

Fonte: Elaborado pelos autores (2026).

Os testes unitários exercitam as regras de negócio sem banco de dados, substituindo os repositórios por dublês: cobrem os serviços de veículos, motoristas, vínculos, despesas,

manutenções, documentos, indicadores, usuários e autenticação, além dos *middlewares* de autenticação, autorização, validação, tratamento de erros, tempo-limite e registro. Os de integração executam contra o banco real e verificam o que somente ele pode responder — se as consultas de fato isolam organizações, se os totais ignoram lançamentos cancelados, se a precisão monetária sobrevive ao percurso completo, se a situação de atraso e a central de alertas usam o mesmo critério das listas e do painel, e se as restrições do esquema recusam o que deveriam. As escritas desse conjunto ocorrem em transações revertidas ao final.

Os testes de ponta a ponta enviam requisições HTTP reais à aplicação montada e conferem a resposta final, passando por rota, autenticação, autorização, controlador, serviço, repositório e banco. Os *tokens* são emitidos pelo próprio Supabase Auth local: o login não é simulado. Cada caso tem um identificador, por área, que aparece no nome do teste; o Quadro 9 apresenta as áreas.

Quadro 9 — Testes de ponta a ponta por área

Área	O que verifica	Casos
Infraestrutura	Saúde da aplicação e do banco, CORS, rotina protegida por segredo	5
Autenticação	*Token* ausente, malformado, forjado, com algoritmo trocado ou sem assinatura; contas pendente e recusada	13
Permissões	Vinte ações negadas e sete permitidas, por papel	27
Isolamento	Dados de outra empresa respondem como inexistentes	8
Recorte do operador	Só os próprios veículos, ficha e lançamentos	6
Veículos	Validação, placa única, vínculo, desativar antes de excluir	7
Despesas	Valor, categoria, data futura, precisão monetária, cancelamento, reativação, histórico	12
Manutenções	Agendar, concluir, cancelar, reativar, atraso calculado	6
Documentos	Anexo por endereço assinado, extensões aceitas, armazenamento fechado	11
Central de alertas	Três frentes, contagem, recorte do operador, sinalização da rotina, CNH vencida no vínculo	5
Cadastro	Empresa nova, cadastro, aprovação, empresa inativa, último administrador	15
Validação	Ordenação fechada, injeção de SQL, identificador inválido, JSON malformado, corpo acima do limite	12
Paginação	Tamanho de página, total no	10

Área	O que verifica	Casos
	cabeçalho, páginas sem repetição	
Total		137

Fonte: Elaborado pelos autores (2026).

A cobertura de código mede quanto do servidor os testes executam. Foi medida com as duas suítes juntas, pois a camada HTTP só é percorrida pela de ponta a ponta. A Tabela 1 apresenta o resultado; as camadas que concentram regras de segurança e de negócio — *middlewares*, rotas e serviços — ficaram acima de 92%.

Tabela 1 — Cobertura de código do servidor

Medida	Cobertura (%)
Linhas	88,4
Funções	89,9
Instruções	86,2
Ramos de decisão	75,9
Linhas em *middlewares*	99,2
Linhas em rotas	98,5
Linhas em serviços	92,3
Linhas em repositórios	85,3
Linhas em controladores	76,5

Fonte: Elaborado pelos autores (2026), rodada de 9 nov. 2026.

Na rodada de 20 de outubro, a interface compilada foi aberta em navegador automatizado por cinco perfis — administrador, gestor, motorista, administrador da plataforma e conta pendente —, com o critério de nenhum erro de tela, de console ou de requisição; as 30 aberturas passaram, cada tela estável entre 0,61 e 0,73 segundo. Em produção, 25 aberturas de tela por quatro perfis passaram sem erro.

5.3 Segurança

Os testes de segurança tentam os ataques conhecidos contra a autenticação e a autorização e verificam que todos são recusados. Treze casos atacam a autenticação: assinatura com outra chave e o mesmo identificador de chave; troca do algoritmo para HS256 com o segredo antigo do serviço, conhecida como confusão de algoritmo; *token* sem assinatura; a chave pública usada como se fosse credencial de usuário; e contas pendentes ou recusadas. As credenciais forjadas recebem 401, e as contas sem aprovação, 403. Vinte e sete casos percorrem a matriz de permissões: cada papel tenta o que não pode e recebe 403. Oito casos tentam ler, alterar e excluir dados de outra empresa e recebem 404, sem revelar que o registro existe. Onze casos enviam entradas hostis — injeção de SQL nos filtros e na

ordenação, identificadores inválidos, JSON malformado e corpo grande demais — e recebem 400, 404 ou 413.

Em produção, a verificação repetiu as recusas principais com contas de demonstração e apenas leituras: o motorista recebeu 403 na lista de usuários, o administrador recebeu 404 ao ler veículo de outra empresa e os *tokens* inválidos receberam 401. O *token* sem assinatura sequer chegou à aplicação: o *firewall* da plataforma o recusou com 403.

5.4 Desempenho

Os testes de carga usam modelo fechado: um número fixo de conexões envia requisições sem pausa entre a resposta e o pedido seguinte. Dez conexões contínuas equivalem a muito mais do que dez pessoas, porque uma pessoa real lê a tela entre um clique e outro. Em todos os cenários, a aplicação foi a versão compilada, com *tokens* reais de cada perfil.

O cenário principal é o *workload* misto, executado em 9 de novembro: todas as rotas disputam a aplicação e o banco ao mesmo tempo, na proporção de um uso realista — 60% de consultas à lista de despesas, 15% de lançamentos de despesa, 8% de consultas a documentos, 8% de verificações de saúde, como faria um monitor externo, 5% de aberturas do painel e 4% de consultas a veículos. Cada requisição sorteia a rota pelo peso e o perfil que a faz. O gerador é um *script* do próprio repositório, que mede o tempo de cada rota dentro do cenário. A máquina tinha processador Apple M5 de dez núcleos e 16 GB de memória, e o gerador, a aplicação e o banco rodaram nela ao mesmo tempo. A Tabela 2 apresenta o resultado por nível de concorrência.

Tabela 2 — Workload misto por nível de concorrência

Conexões	Requisições	Req/s	Mediana (ms)	p90 (ms)	p99 (ms)	Erros
10	43.574	1.452	6,2	11,3	17,2	0
50	30.796	1.027	44,2	73,0	109,6	0
100	23.697	790	117,5	189,9	285,0	0
100, base limpa	26.686	890	102,1	168,4	230,1	0

Fonte: Elaborado pelos autores (2026), rodada de 9 nov. 2026, registrada em docs/testes/workload-misto.json.
Nota: Cada nível durou 30 s, após 3 s de aquecimento. Os três primeiros níveis rodaram em sequência, na mesma base; a última linha repete o nível de 100 conexões numa base sem as despesas gravadas pelos níveis anteriores.

Nenhuma das 124.753 requisições falhou. A vazão cai à medida que a concorrência sobe, por dois motivos. O primeiro é a fila: a aplicação mantém dez conexões com o banco, e acima disso as requisições esperam a vez, o que transforma concorrência em tempo de

resposta sem aumentar a vazão. O segundo é o próprio cenário: os lançamentos gravaram 16.374 despesas durante a rodada, e as consultas seguintes percorreram uma base maior. Repetido numa base limpa, o nível de 100 conexões atendeu 890 requisições por segundo, contra 790 na sequência; a diferença é o efeito do volume, e o restante, o da fila. A Tabela 3 detalha cada rota.

Tabela 3 — Workload misto por rota

Rota	Participação (%)	Mediana, 10 con. (ms)	p99, 10 con. (ms)	Mediana, 100 con. (ms)	p99, 100 con. (ms)
GET /api/expenses	60,3	6,0	11,6	112,4	188,3
POST /api/expenses	14,8	11,9	20,5	194,2	342,5
GET /api/documents	7,9	5,7	12,0	124,2	248,8
GET /health	8,0	1,5	4,9	53,0	93,8
GET /api/dashboard/indicators	4,9	7,4	14,8	130,8	259,4
GET /api/vehicles	4,1	5,5	11,8	121,4	251,5

Fonte: Elaborado pelos autores (2026), rodada de 9 nov. 2026, registrada em docs/testes/workload-misto.json.
Nota: A participação é a do nível de 10 conexões; nos demais, ficou a menos de um ponto percentual do peso definido.

O lançamento de despesa é a operação mais lenta do cenário, e é esperado que seja: ele grava a despesa e o registro de histórico na mesma transação. O painel, que soma despesas, manutenções e documentos do período, ficou próximo das consultas simples, porque as somas são feitas no banco. Mesmo com cem conexões, o pior p99 foi de 342,5 ms, abaixo do limite de um segundo do RNF04.

A rodada de 20 de outubro mediu cada rota isolada, com dez conexões por dez segundos, conforme a Tabela 4. Os números não são comparáveis aos do *workload* misto: a máquina era outra, e a rota testada isolada não disputa recursos com as demais.

Tabela 4 — Carga por rota isolada

Rota	Conexões	Req/s	Mediana (ms)	p99 (ms)	Erros
GET /health	50	4.677	9	24	0
GET /api/documents	10	899	10	20	0
GET /api/vehicles	10	874	10	21	0
GET /api/expenses, motorista	10	718	13	25	0
GET /api/users/me	10	648	14	26	0
GET /api/expenses,	10	533	17	38	0

Rota	Conexões	Req/s	Mediana (ms)	p99 (ms)	Erros
administrador POST /api/expenses	10	506	18	34	0
GET /api/dashboard/indicators	10	408	23	38	0

Fonte: Elaborado pelos autores com base no Relatório de Testes do Fleet Manager (2026), rodada de 20 out. 2026.

O painel, a rota mais cara, foi também submetido a estresse, com a concorrência subindo de 10 a 200 conexões, conforme a Tabela 5. Trata-se de uma única rota em quatro níveis de carga, e não de rotas diferentes. A vazão permaneceu estável em torno de 400 painéis por segundo, e o aumento de conexões se converteu em tempo de espera, sem erro.

Tabela 5 — Estresse do painel de indicadores

Conexões	Req/s	Mediana (ms)	Erros
10	408	23	0
50	421	117	0
100	427	230	0
200	402	473	0

Fonte: Elaborado pelos autores com base no Relatório de Testes do Fleet Manager (2026), rodada de 20 out. 2026.
Nota: Com 200 conexões, o p99 foi de 601 ms.

O teste de volume verificou se o tempo de resposta depende de quanto a empresa acumulou, com 50 mil despesas em uma única empresa, distribuídas por um ano. A Tabela 6 mostra que não: a lista é paginada, de modo que a resposta tem tamanho fixo, cerca de 28 KB, com 500 ou com 50 mil lançamentos, e o painel devolve só os totais.

Tabela 6 — Tempos com 50 mil despesas em uma empresa

Operação	Tempo
Lista de despesas, primeira página	17 ms
Lista de despesas, última página	30 ms
Lista com filtro dos últimos 30 dias	11 ms
Painel, período padrão de 180 dias	21 ms
Tela de despesas aberta no navegador	0,69 s
Painel aberto no navegador	0,75 s

Fonte: Elaborado pelos autores com base no Relatório de Testes do Fleet Manager (2026), rodada de 20 out. 2026.

Em produção, a carga foi moderada, para não consumir a cota do plano gratuito, e se limitou a leituras. A Tabela 7 apresenta o resultado. Cerca de 70 ms da mediana de 90 ms correspondem ao trajeto de rede. Ao passar de 10 para 25 conexões no painel, a vazão subiu de 103 para 244 requisições por segundo e a mediana não mudou: a plataforma abre novas instâncias conforme a demanda, em vez de enfileirar as requisições.

Tabela 7 — Carga em produção

Rota	Conexões	Req/s	Mediana (ms)	p99 (ms)	Erros
GET /api/vehicles	10	105	90	228	0
GET /api/expenses	10	106	89	227	0
GET /api/dashboard/indicators	10	103	91	268	0
GET /api/dashboard/indicators	25	244	91	396	0

Fonte: Elaborado pelos autores com base no Relatório de Testes do Fleet Manager (2026), rodada de 20 out. 2026.
Nota: Total de 11.840 requisições, todas com resposta 200.

5.5 Resiliência

O teste de resiliência pausou o banco de dados com a aplicação no ar e depois o retomou, para observar o comportamento diante de uma dependência fora do ar. A Tabela 8 apresenta as respostas.

Tabela 8 — Comportamento com o banco fora do ar e na volta

Momento e requisição	Resposta	Tempo
Banco fora, GET /health	503, banco indisponível	3,0 s
Banco fora, GET /api/vehicles	503, serviço indisponível	10,0 s
Banco fora, GET /api/vehicles sem *token*	401	1 ms
Banco de volta, GET /health	200	4 ms
Banco de volta, GET /api/vehicles	200	7 ms

Fonte: Elaborado pelos autores com base no Relatório de Testes do Fleet Manager (2026), rodada de 20 out. 2026.

O cliente recebe 503, o código padrão para serviço temporariamente indisponível, em vez de um erro genérico ou de uma conexão presa; o processo continua atendendo o que não depende do banco, como a recusa de quem não tem credencial; e, quando o banco volta, a primeira requisição já é atendida, sem reiniciar nada. Um monitor externo consultando o /health detecta a queda em três segundos.

5.6 Atendimento dos objetivos específicos

O Quadro 10 relaciona cada objetivo específico ao que foi entregue e à evidência que o comprova.

Quadro 10 — Objetivos específicos, entregas e evidências

Objetivo	Entrega	Evidência
Cadastro de veículos, motoristas e usuários, com vínculo	Módulos de veículos e motoristas; vínculo com período; aprovação de contas	Testes de ponta a ponta de veículos e de cadastro; seções 4.4 a 4.6
Despesas vinculadas a veículo, em seis categorias	Módulo de despesas, com cancelamento e paginação	Testes de despesas e de integração (precisão monetária); RN01, RN15, RN16
Manutenções preventivas e corretivas, com seus estados	Quatro situações, com o atraso calculado pela data prevista	Testes de manutenção; RN09 a RN11
Documentos com vencimento, situação automática, anexo e central de alertas	Módulo de documentos; central com três frentes; rotina diária	Testes de documentos e da central; Figura 9; RN12 e RN13
Painel de indicadores	Custo total, médio, por veículo, mensal e por categoria	Figura 8; Tabelas 5 e 6
Autenticação e controle de acesso por papel	Supabase Auth com ES256; RBAC por empresa; recorte do operador	Seção 5.3; Quadro 4
Verificação da qualidade em três níveis, em integração contínua, e avaliação de segurança, desempenho e resiliência	402 testes automatizados; integração contínua; carga, volume e resiliência	Seções 5.2 a 5.5

Fonte: Elaborado pelos autores (2026).

5.7 Limitações e discussão

A principal limitação do trabalho é de natureza metodológica: o sistema não foi aplicado em uma organização real. A validação empregou base de dados fictícia, ainda que consistente, gerada por rotina automatizada e com volume de um ano de operação. Em consequência, os benefícios pretendidos — redução de sanções por documentação vencida, migração da manutenção corretiva para a preventiva e visibilidade do custo por veículo — permanecem como expectativas fundamentadas, e não como resultados medidos. A comprovação empírica demandaria adoção por uma frota real, com medição comparativa antes e depois, o que se situa fora do escopo temporal deste trabalho.

Registra-se, na mesma linha, que a validação das funcionalidades com especialistas do domínio não foi realizada. O procedimento previsto consiste em demonstração guiada do sistema seguida de roteiro estruturado de perguntas, aplicado a gestores de frota e proprietários de operações de transporte, com registro do perfil de cada respondente e sem coleta de dado pessoal identificável. O procedimento está descrito por permitir sua execução, não por ter sido executado — nenhuma entrevista foi conduzida até o fechamento deste texto.

Quanto aos testes, três limites devem ser considerados na leitura dos resultados. Primeiro, a verificação das telas no navegador foi executada, mas o *script* que a realiza não está no repositório nem na integração contínua: a verificação automatizada permanente

alcança o servidor, e não a camada de apresentação. Segundo, nos testes de carga locais, o gerador, a aplicação e o banco dividiram a mesma máquina, de modo que os números absolutos dependem do equipamento e servem para comparar cenários, e não para prever a capacidade da produção. Terceiro, a produção só recebeu carga moderada e de leitura.

Dois problemas encontrados na revisão foram corrigidos antes do fechamento deste texto. O primeiro era a publicação em paralelo com os testes, tratada na seção 4.11. O segundo foi uma vulnerabilidade alta publicada para a biblioteca de validação depois da rodada de outubro — consumo excessivo de memória ao validar coleções muito grandes —, corrigida pela migração do Zod para a versão 4, com os 402 testes aprovados após a troca; na mesma ocasião, as vulnerabilidades moderadas em ferramentas de desenvolvimento foram eliminadas, e a auditoria passou a não apontar nenhuma.

Do ponto de vista funcional, registram-se as seguintes limitações. A verificação de posse do endereço de correio eletrônico está desativada no cadastro, decorrência da exclusão do envio de mensagens do escopo; o risco é mitigado pela aprovação manual exigida antes de qualquer acesso, e a mesma dependência impede a recuperação autônoma de senha. A validade da habilitação existe em dois lugares — na ficha do motorista e no documento do tipo CNH —, e o sistema não as sincroniza; a central de alertas evita o aviso em dobro, mas a unificação é evolução natural do modelo. A integração com sistemas governamentais — consulta de pontuação em habilitação, por exemplo — não foi implementada, por depender de acesso não disponível publicamente. E não há planejamento nem otimização de rotas, recurso deliberadamente fora do escopo.

Sobre a escolha de acesso a dados, registra-se a contrapartida assumida: prescindir de mapeador objeto-relacional aumenta a verbosidade das operações rotineiras e transfere ao desenvolvedor a responsabilidade de declarar, em cada consulta, a condição de isolamento por organização. Optou-se por essa forma para que tal condição permaneça visível no ponto em que é aplicada, e a verificação automatizada contra o banco real cobre justamente esse risco.

Cada uma dessas limitações foi identificada por verificação do próprio código e dos testes, e não inferida. O registro explícito cumpre função metodológica: delimita o que o sistema efetivamente entrega e distingue-o do que permanece como evolução possível.

6 CONCLUSÃO

Este trabalho apresentou o Fleet Manager, sistema web para gestão de frotas que centraliza o controle operacional, financeiro e documental de veículos em uma base de dados relacional única. O objetivo geral foi alcançado: o sistema foi desenvolvido, verificado e publicado em ambiente de produção, e os dez requisitos funcionais definidos no termo de abertura foram atendidos.

Os objetivos específicos foram cumpridos conforme o Quadro 10. O cadastro de veículos, motoristas e usuários, com vínculo de período, as despesas em seis categorias e o painel de indicadores estão em produção. O controle de manutenções acompanha quatro situações, e o atraso é apurado pela data prevista, sem depender de alguém marcá-lo. Os documentos têm situação calculada, anexo e uma central de alertas que reúne, numa só tela, documentos, manutenções e habilitações que exigem ação. O controle de acesso separa empresas e papéis no servidor, e os ataques testados foram recusados. A qualidade foi verificada por 402 testes automatizados em três níveis, executados em integração contínua, e por testes de carga, volume e resiliência que mantiveram o percentil 99 abaixo de um segundo em todos os cenários e não registraram erro.

A contribuição técnica do trabalho concentra-se em cinco decisões. A primeira é a organização do servidor em camadas com dependência unidirecional, que não se limita a uma escolha estética: por isolarem-se do protocolo HTTP, as regras de negócio puderam ser exercitadas por testes sem infraestrutura de apoio. A segunda é a delegação da autenticação a serviço especializado com verificação por chave pública, mantendo, contudo, a autorização sob controle da aplicação — desenho que retira do sistema a responsabilidade de guardar senhas sem transferir a terceiros a autoridade sobre permissões. A terceira é o compartilhamento de tipos entre interface e servidor, que converte divergências de contrato em erros de compilação. A quarta é a segregação de dados por organização aplicada na camada de aplicação, com o identificador da empresa presente em cada entidade operacional e derivado sempre do perfil autenticado. A quinta é o cálculo das situações de prazo a cada consulta, em vez de gravá-las: documentos, manutenções e habilitações não ficam com situação defasada porque ninguém precisa atualizá-la.

O percurso também produziu aprendizado sobre a distância entre projetar e publicar, e entre documentar e verificar. Duas exigências só se manifestaram na publicação: o pacote compartilhado precisou passar a emitir código compilado, e a rotina agendada precisou migrar para o agendador da plataforma. A etapa de testes, por sua vez, mostrou que documento e

sistema haviam divergido: o documento prometia o atraso de manutenções e uma central de alertas mais ampla do que o sistema entregava, e o sistema já tinha testes, integração contínua e medições que o documento não registrava. A revisão final adotou, em cada ponto, a versão mais completa, e as duas partes voltaram a corresponder.

Como delimitação, reitera-se que o sistema não foi aplicado em organização real, razão pela qual os benefícios pretendidos permanecem como expectativas fundamentadas. A continuidade natural do trabalho compreende a validação em frota real, com medição comparativa; a inclusão dos testes de tela na integração contínua; e a unificação da validade da habilitação entre a ficha do motorista e o documento.

REFERÊNCIAS

FIELDING, Roy Thomas. **Architectural Styles and the Design of Network-based Software Architectures**. 2000. Tese (Doutorado em Ciência da Computação) — University of California, Irvine, 2000.

FLEURY, Paulo Fernando; WANKE, Peter; FIGUEIREDO, Kleber Fossati (org.). **Logística empresarial**: a perspectiva brasileira. São Paulo: Atlas, 2000.

GITHUB. **GitHub Actions documentation**. 2026. Disponível em: https://docs.github.com/actions. Acesso em: 9 nov. 2026.

HUMBLE, Jez; FARLEY, David. **Continuous delivery**: reliable software releases through build, test, and deployment automation. Upper Saddle River: Addison-Wesley, 2010.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 25010**: systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — system and software quality models. Geneva: ISO, 2011.

JONES, Michael B.; BRADLEY, John; SAKIMURA, Nat. **JSON Web Token (JWT)**. RFC 7519. Internet Engineering Task Force, maio 2015. Disponível em: https://datatracker.ietf.org/doc/html/rfc7519. Acesso em: 26 ago. 2026.

KRUCHTEN, Philippe. The "4+1" View Model of Software Architecture. **IEEE Software**, v. 12, n. 6, p. 42-50, nov. 1995.

META. **React**: The library for web and native user interfaces. Meta Open Source, 2026. Disponível em: https://react.dev. Acesso em: 26 ago. 2026.

MICROSOFT. **TypeScript Documentation**. 2026. Disponível em: https://www.typescriptlang.org/docs. Acesso em: 26 ago. 2026.

NIELSEN, Jakob. **Usability engineering**. Boston: Academic Press, 1993.

OPENJS FOUNDATION. **Node.js Documentation**. 2026. Disponível em: https://nodejs.org/docs. Acesso em: 26 ago. 2026.

PORSAGER, Rasmus. **postgres**: PostgreSQL client for Node.js. 2026. Disponível em: https://github.com/porsager/postgres. Acesso em: 21 set. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL Documentation**. 2026. Disponível em: https://www.postgresql.org/docs. Acesso em: 26 ago. 2026.

SANDHU, Ravi S. *et al.* Role-based access control models. **IEEE Computer**, v. 29, n. 2, p. 38-47, fev. 1996.

SOMMERVILLE, Ian. **Engenharia de Software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011.

SUPABASE. **Supabase Documentation**. 2026. Disponível em: https://supabase.com/docs. Acesso em: 26 ago. 2026.

TAILWIND LABS. **Tailwind CSS Documentation**. 2026. Disponível em: https://tailwindcss.com/docs. Acesso em: 26 ago. 2026.

VERCEL, INC. **Vercel Documentation**. 2026. Disponível em: https://vercel.com/docs. Acesso em: 26 ago. 2026.

VITE TEAM. **Vite Documentation**. 2026. Disponível em: https://vite.dev. Acesso em: 26 ago. 2026.

VITEST. **Vitest**: a Vite-native testing framework. 2026. Disponível em: https://vitest.dev. Acesso em: 26 ago. 2026.

ZOD. **Zod**: TypeScript-first schema validation with static type inference. 2026. Disponível em: https://zod.dev. Acesso em: 26 ago. 2026.

GLOSSÁRIO

Dublê de teste: objeto que substitui uma dependência real durante o teste, como um repositório que devolve dados fixos em vez de consultar o banco.

Entrega contínua: prática de manter toda versão aprovada pronta para publicação, com a publicação automatizada.

Firewall: *filtro de rede posicionado à frente da aplicação, que recusa requisições suspeitas antes que cheguem a ela*

Idempotente: operação que, executada mais de uma vez, produz o mesmo efeito que executada uma só vez.

Integração contínua: prática de compilar e testar automaticamente cada alteração enviada ao repositório.

Middleware: *função que intercepta a requisição antes do controlador, para autenticar, autorizar, validar ou registrar*

Migration: *arquivo que descreve uma alteração do esquema do banco, aplicado em ordem e uma única vez*

Modelo de carga fechado: teste de carga em que um número fixo de conexões envia requisições sem pausa entre uma resposta e o pedido seguinte.

Percentil: valor abaixo do qual está uma fração das medidas: o p50, ou mediana, deixa metade abaixo; o p90, 90%; o p99, 99%.

Recorte: conjunto de registros que um perfil alcança: a empresa inteira, para administrador e gestor; os próprios lançamentos e veículos vinculados, para o operador.

Token: *credencial assinada que o cliente apresenta a cada requisição para provar quem é*

Vazão: quantidade de requisições atendidas por segundo.

Workload: *proporção entre os tipos de requisição de um teste de carga, que reproduz um padrão de uso*

APÊNDICE A — Termo de abertura do projeto

A.1 Identificação do projeto

Nome do projeto: Fleet Manager — Sistema Web de Gestão de Frotas. Código do projeto: FM-PFI-2026.

A.2 Objetivos

Desenvolver um sistema web para gestão de frotas, com foco no controle operacional, financeiro e documental de veículos, permitindo transparência dos custos por ativo, redução de riscos legais e apoio à tomada de decisão.

A.3 Escopo

O projeto contempla o cadastro e a gestão de veículos e motoristas; o registro de despesas operacionais vinculadas a cada veículo; o controle de manutenções preventivas e corretivas, com acompanhamento de estado; o controle de documentos obrigatórios, com classificação de vencimentos; o painel de indicadores financeiros; e os mecanismos de autenticação e controle de acesso por papel.

Não fazem parte do escopo o rastreamento por satélite em tempo real, a telemetria avançada, o planejamento e a otimização de rotas, a integração automática com sistemas governamentais, o desenvolvimento de aplicativo móvel nativo e a notificação de vencimentos por canais externos.

A.4 Cronograma resumido

O Quadro 11 apresenta o cronograma resumido do projeto.

Quadro 11 — Cronograma resumido

Fase	Marco	Período
Iniciação	Projeto aprovado	Fevereiro de 2026
Planejamento	Requisitos e arquitetura definidos	Março de 2026
Execução	Servidor e modelo de dados concluídos	Abril de 2026
Execução	Interface e módulos funcionais concluídos	Maio de 2026
Execução	Painel de indicadores e gestão de usuários	Junho de 2026

Fase	Marco	Período
Monitoramento	Produto mínimo viável verificado	Junho de 2026
Execução	Migração da autenticação e publicação em produção	Agosto de 2026
Encerramento	Entrega e apresentação	Agosto de 2026
Execução	Segregação por organização, administração da plataforma e anexos servidos pela aplicação	Setembro de 2026
Monitoramento	Testes de ponta a ponta, integração contínua, carga e resiliência	Outubro de 2026
Monitoramento	Revisão das regras de negócio e *workload* misto	Novembro de 2026
Encerramento	Entrega do documento revisado e defesa	Novembro de 2026

Fonte: Elaborado pelos autores (2026).

A.5 Partes interessadas

O Quadro 12 relaciona as partes interessadas e suas responsabilidades.

Quadro 12 — Partes interessadas do projeto

Participante	Responsabilidade	Autoridade
Professor orientador	Orientação, acompanhamento e avaliação do projeto	Alta
André Luiz Cavalcante da Silva	Desenvolvimento, modelagem e documentação	Alta
Luiz Eduardo Pacheco	Desenvolvimento, modelagem e documentação	Alta
Gregory Figueiredo de M. P. Jereissati	Desenvolvimento, modelagem e documentação	Alta
Gestores de frota (usuários finais)	Utilização estratégica e validação do sistema	Média
Operadores (usuários finais)	Registro e atualização das informações operacionais	Baixa

Fonte: Elaborado pelos autores (2026).

A.6 Restrições

- Orçamento nulo: o projeto restringe-se a serviços com camada gratuita permanente, condição que determinou a escolha das plataformas de hospedagem.
- Equipe de três integrantes, em regime acadêmico, com dedicação parcial.
- Prazo delimitado pelo calendário do Projeto Final Integrador I.
- Ausência de acesso a uma frota real para validação empírica.

A.7 Riscos e mitigação

- Dependência de plataformas de terceiros. Materializou-se durante o projeto: o ambiente de banco de dados foi perdido e precisou ser integralmente recriado. Mitigação adotada: versionamento do esquema por migrações e de scripts de configuração, tornando o ambiente reconstruível a partir do repositório.
- Alteração das condições comerciais dos serviços utilizados. Materializou-se na plataforma inicialmente prevista para o servidor, que deixou de oferecer camada gratuita permanente. Mitigação adotada: substituição por alternativa gratuita.
- Divergência entre a documentação e o sistema implementado. Materializou-se entre setembro e novembro, quando o sistema evoluiu além do documento. Mitigação adotada: verificação de cada afirmação técnica contra o código-fonte e os testes antes do registro, e revisão final item a item, descrita na seção 3.1.
- Concentração de conhecimento em um integrante. Mitigação adotada: revisão conjunta e documentação das decisões arquiteturais.

A.8 Critérios de sucesso

- Atendimento dos dez requisitos funcionais definidos no levantamento.
- Sistema publicado e operante em ambiente de produção, acessível por navegador.
- Compilação sem erros e aprovação integral dos testes automatizados das regras de negócio.
- Ambiente reconstruível a partir do repositório, sem dependência de configuração não documentada.
- Documentação técnica correspondente ao sistema efetivamente implementado.

APÊNDICE B — Documento de arquitetura de software

Este apêndice descreve as decisões arquiteturais do Fleet Manager segundo o modelo de visões "4+1" (Kruchten, 1995), que organiza a descrição de uma arquitetura em visões complementares, cada uma dirigida a um público distinto.

B.1 Requisitos e restrições arquiteturais

O Quadro 13 relaciona os requisitos e as restrições arquiteturais e a solução adotada para cada um.

Quadro 13 — Requisitos e restrições arquiteturais

Requisito	Solução adotada
Linguagem	TypeScript no servidor e na interface, com tipos compartilhados entre ambos.
Plataforma	Interface estática e camada de serviço publicadas em projeto único na Vercel, no mesmo domínio.
Segurança	Autenticação delegada ao Supabase Auth, com *token* ES256 verificado pela chave pública do serviço; autorização por papéis aplicada na aplicação.
Persistência	PostgreSQL gerenciado pelo Supabase, acessado por SQL parametrizado a partir da camada de repositórios.
Armazenamento de arquivos	Supabase Storage, alcançado apenas pelo servidor, que emite endereços assinados de validade curta após aplicar o controle de acesso.
Internacionalização	Português brasileiro e inglês, com preferência persistida no navegador.
Qualidade	Testes em três níveis executados em integração contínua no GitHub Actions; publicação na Vercel apenas após a aprovação de todos eles.
Custo	Restrição a serviços com camada gratuita permanente.

Fonte: Elaborado pelos autores (2026).

B.2 Visão de casos de uso

Apresentada na Figura 2, seção 4.1. Os casos significativos para a arquitetura são a autenticação, que atravessa todas as rotas protegidas, e a gestão de usuários, que determina o papel aplicado nas demais operações.

B.3 Visão lógica

Apresentada na Figura 3, seção 4.2. A aplicação organiza-se em camadas com dependência unidirecional. As classes mais significativas concentram-se na camada de serviços, que materializa as regras de negócio descritas no Quadro 7, e na de repositórios, que isola o acesso ao banco. Os *middlewares* de autenticação, autorização e validação atuam de forma transversal às rotas protegidas.

B.4 Visão de implementação

Apresentada na Figura 4, seção 4.3. O código distribui-se em duas aplicações e um pacote compartilhado, este último contendo as enumerações e os objetos de transferência de dados utilizados por ambas. A Figura 6 detalha a realização do caso de uso de autenticação, e a Figura 10, a do anexo de arquivo a documento.

B.5 Visão de processo

O sistema apresenta dois fluxos de execução distintos. O primeiro é síncrono e orientado a requisição: cada chamada é atendida de forma independente, sem estado compartilhado entre requisições, o que se coaduna com o modelo de execução sem servidor dedicado adotado na publicação. Cada requisição tem tempo-limite de dez segundos e gera uma linha de registro. O segundo é a rotina diária de sinalização de vencimentos, disparada pelo agendador da plataforma e projetada para ser idempotente, de modo que execuções repetidas não produzam efeito colateral; ela registra a data da sinalização, exibida na central de alertas.

B.6 Visão de implantação

Apresentada na Figura 11, seção 4.10. A topologia compreende três nós: o dispositivo do usuário, executando o navegador; o projeto na plataforma de nuvem, que atende tanto os artefatos estáticos quanto a camada de serviço; e a plataforma de dados, que provê banco relacional, autenticação e armazenamento de arquivos.

B.7 Visão de dados

Apresentada na Figura 5, seção 4.4, e detalhada no Quadro 3. O esquema é versionado por migrações incrementais, o que permite reconstruir integralmente a estrutura relacional a partir do repositório. A proteção do banco contra acesso externo é descrita na seção 4.9.

APÊNDICE C — Roteiro para reproduzir os testes

Os testes podem ser repetidos em qualquer máquina com Node.js e um ambiente de contêineres, a partir da raiz do repositório. O Supabase local é criado do zero e fica isolado da produção. Os comandos, em ordem:

- gerar a chave de assinatura local: npx supabase gen signing-key --algorithm ES256 --append, sobre um arquivo supabase/signing_keys.json vazio;
- subir o Supabase local: npx supabase start;
- criar o banco e os dados fictícios: npm run db:migrate e npm run db:seed;
- suíte padrão, com os testes unitários e de integração: npm run test:api;
- suíte de ponta a ponta: npm run test:e2e;
- volume de demonstração, com um ano de operação: npm run db:popular, desfeito por npm run db:popular -- --remover;
- cobertura das duas suítes juntas: npx vitest run --config vitest.cobertura.config.mts --coverage, na pasta apps/api;
- *workload* misto, com a aplicação compilada no ar: npm run carga:workload, na pasta apps/api, que apaga ao final as despesas que criou;
- análise estática: npm run lint e npx tsc --noEmit em cada aplicação.

A integração contínua executa os mesmos passos, na mesma ordem, no arquivo .github/workflows/testes.yml. Os resultados do *workload* ficam gravados em docs/testes/, um arquivo por rodada, com data, máquina, pesos das rotas e medidas de cada nível.